



**COMSATS University Islamabad,
Sahiwal Campus, Pakistan**

Alhewan App

**A Project Presented To
COMSATS University, Islamabad
Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2022-2026)

By

Muhammad Ammaz Rehan	CIIT/SP22-BCS-065/SWL
Muhammad Taha Zafar	CIIT/SP22-BCS-093/SWL
Tuba Naz	CIIT/SP22-BCS-111/SWL

DECLARATION

We hereby declare that this system, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this app and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Ammaz Rehan

Muhammad Taha Zafar

Tuba Naz

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “**Alhewan App**” was developed by **Muhammad Ammaz Rehan (CIIT/SP22-BCS-065/SWL)**, **Muhammad Taha Zafar (CIIT/SP22-BCS-093/SWL)** and **Tuba Naz (CIIT/SP22-BCS-111/SWL)** under the supervision of “**Sir. Arslan Sarwar**” and co supervisor “**Sir. Jamil**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor
Mr. Arslan Sarwar
Lecturer

Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

External Examiner
Dr. Muhammad Munawar Iqbal
Assistant Professor
Department of Computer Science
UET Taxila

Head of Department
Dr. Zafar Iqbal Roy
Assistant Professor
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

The Alhewan App represent an advanced and innovative digital solution designed to improve and modernize the veterinary healthcare experience for both pet owners and veterinarians. Developed using Flutter for mobile applications and a web-based admin panel, the system provides a unified platform for online consultations, appointment scheduling, pet medical history management, product purchasing, and real-time communication. By addressing the challenges of traditional veterinary services such as lack of accessibility, limited record-keeping, and inefficient appointment processes Alhewan aims to create a fast, reliable, and user-friendly digital ecosystem. At its core, the project integrates three major components: the User Mobile App, the Doctor/Veterinarian Mobile App, and the Admin Management Panel. Together, these modules offer a complete solution for managing veterinary services. Users can conveniently book appointments, purchase medicines and pet products, communicate with verified doctors through real-time chat, and maintain digital medical records of their pets. For veterinarians, the system simplifies daily operations by providing access to past medical history, appointment management, patient communication, earning insights, and profile verification. The Admin Panel ensures smooth system operations by managing doctors' verification, inventory, orders, appointments, and user activities. The strengths of the system lie in its reliability, interactive user interface, personalized features, and seamless cross-module integration. Pet owners can easily navigate categories such as vaccines, deworming, supplements, and grooming tools, while veterinarians benefit from accurate patient history, secured communication, and automated appointment tracking. Real-time order management and inventory monitoring further enhance the platform's operational efficiency. The project scope includes well-structured modules such as user authentication, product management, appointment booking, medical record tracking, doctor verification, online consultation, chat system, and payment integration. Each module has undergone thorough testing including manual testing, system testing, functional testing, and integration testing to ensure smooth performance across all platforms. The system adheres to software development standards, providing stability, scalability, and high usability. Looking ahead, Alhewan acknowledges possible limitations such as multilingual support requirements, scalability challenges with higher traffic, and expanding inventory management functionalities. These areas will be addressed through future enhancements and continuous improvement strategies.

Acknowledgment

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Mr. Muhammad Arslan Sarwar” and our co Supervisor “Mr. Muhammad Jamil”. Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Ammaz Rehan

Muhammad Taha Zafar

Tuba Naz

Abbreviations

Abbreviation	Full Form
SRS	Software Requirements Specification
SDD	Software Design Description
UI	User Interface
UX	User Experience
API	Application Programming Interface
FCM	Firebase Cloud Messaging
CRUD	Create, Read, Update, Delete
SDLC	Software Development Life Cycle
ERD	Entity Relationship Diagram
OTP	One-Time Password
NFR	Non-Functional Requirements
FR	Functional Requirements
DB	Database
APK	Android Package Kit

Table of Contents

1. Introduction.....	1
1.1 Project Brief	2
1.2 Relevance to the Course Module	2
1.2.1 Requirement Engineering	3
1.2.2 System Design and Modelling.....	3
1.2.3 Software Development Life Cycle (SDLC)	3
1.2.4 Database and Backend Concepts	4
1.2.5 User Interface Design	4
1.2.6 Testing and Quality Assurance.....	4
1.3 Background	5
1.4 Literature Review.....	5
1.4.1 Summary of Existing Veterinary Platform.....	6
1.5 Methodology and Software Life Cycle	7
1.5.1 Requirement Analysis.....	8
1.5.2 Planning & Iteration Design	9
1.5.3 Design Phase.....	9
1.5.4 Implementation.....	9
1.5.5 Testing	9
1.5.6 Deployment	10
1.5.7 Feedback & Review.....	10
1.6 Rationale Behind Methodology	10
2. Problem Definition.....	12
2.1 Problem Statement	12
2.2 Deliverables and Development Requirements	13
2.2.1 Project Deliverables.....	13
2.2.2 Development Requirements	15
2.3 Current System.....	16
3. Requirement Analysis.....	18
3.1 Use Case Diagram.....	18
3.1.1 Use Case Diagram of Alhewan App.....	18
3.2 Use Case Descriptions.....	20
3.2.1 Register / Login Use Case	20

3.2.2 Search Veterinarians Use Case.....	21
3.2.3 View Veterinarian Profile Use Case.....	22
3.2.4 Book Appointment Use Case	22
3.2.5 Online Payments Use Case.....	23
3.2.6 Manage Pet Health Records Use Case	24
3.2.7 Submit Review and Rating Use Case	25
3.2.8 Search Medicines / Products Use Case.....	26
3.2.9 View Medicine / Product Details.....	26
3.2.10 Add Item to Cart Use Case	27
3.2.11 Place Order Use Case	28
3.2.12 Pay Online for Order	29
3.3 Functional Requirements.....	29
3.3.1 Customer Module	30
3.3.2 Veterinarian Module.....	30
3.3.3 Admin Module.....	31
3.3.4 Appointment Booking Module.....	31
3.3.5 Pet Health Tracking Module.....	31
3.3.6 Payment & Billing Module.....	31
3.3.7 Review & Feedback Module (Customer Only).....	31
3.4 Non-Functional Requirements	32
3.4.1 Usability.....	32
3.4.2 Performance.....	32
3.4.3 Security	32
3.4.4 Portability	33
3.4.5 Reliability	33
4. Design and Architecture	35
4.1 System Architecture	35
4.2 Process Flow	37
4.3 Design Models.....	40
4.3.1 Sequence Diagram.....	40
4.4 Class Diagram	46
4.5 Data Design	48
5. Implementation	51

5.1 Algorithm	53
5.1.1 User Authentication Algorithm	53
5.1.2 Pet Profile Algorithm.....	55
5.1.3 Appointment Booking Algorithm.....	56
5.1.4 Manual Payment & Screenshot Upload Algorithm.....	57
5.1.5 Review Submission Algorithm.....	58
5.1.6 Veterinarian Verification Algorithm	59
5.2 External APIs	60
5.3 User Interface	61
5.3.1 Screenshot of Login Screen.....	63
5.3.2 Screenshot of Sign-Up Screen.....	64
5.3.3 Screenshot of User Home Screen	65
5.3.4 Screenshot of Category Screen.....	66
5.3.5 Screenshot of User Profile Screen	67
5.3.6 Screenshot of Doctor Detail Screen.....	68
5.3.7 Screenshot of Booking Appointments Screen	69
5.3.8 Screenshot of Admin Home Screen.....	70
5.3.9 Screenshot of Admin Side Product Screen.....	71
5.3.10 Screenshot of Order Management Screen	72
5.3.11 Screenshot of Doctor Verification Management.....	73
5.3.12 Screenshot of Appointment Confirmation Screen.....	74
5.3.13 Screenshot of Doctor Profile Screen	75
5.3.14 Screenshot of Doctor Chat Screen.....	76
5.3.15 Screenshot of Doctor Side Pet History Screen	77
5.3.16 Screenshot of Doctor Side Appointments Payment Screen.....	78
6. Testing and Evaluation	80
6.1 Manual Testing.....	81
6.1.1 Manual Testing User Application.....	81
6.1.2 Manual Testing Doctor Application	82
6.1.3 Manual Testing Admin Panel	83
6.2 System Testing	85
6.3 Unit Testing.....	86
6.3.1 Unit Test Appointment Booking	86

6.3.2 Unit Test Product Stock Management.....	86
6.3.3 Unit Test Real Time Chat.....	87
6.4 Functional Testing.....	88
6.5 Integration Testing	89
7. Conclusion and Future Work.....	92
7.1 Conclusion.....	92
7.2 Limitations	92
7.3 Future Work	92
7.4 Final Remarks	93
References.....	94

List of Figures

Figure 1.1 Life Cycle of Agile Methodology	8
Figure 3.1 Use case diagram of Alhewan App	19
Figure 4.1 System Architecture of Alhewan App.....	37
Figure 4.2 Process Flow of Alhewan App	39
Figure 4.3 Sequence Diagram of Alhewan App	40
Figure 4.4 User Authentication Module Sequence Diagram	41
Figure 4.5 Appointment Booking Sequence Diagram	43
Figure 4.6 Vet Approval / Verification Sequence Diagram	44
Figure 4.7 Veterinarian Module Sequence Diagram	45
Figure 4.8 Class Diagram of Alhewan App	48
Figure 5.1 Screenshot of Login Screen.....	63
Figure 5.2 Screenshot of Sign-Up Screen.....	64
Figure 5.3 Screenshot of User Home Screen.	65
Figure 5.4 Screenshot of Category Screen.....	66
Figure 5.5 Screenshot of User Profile Screen.....	67
Figure 5.6 Screenshot of Booking Appointment Screen.	68
Figure 5.7 Screenshot of My Appointments Screen.	69
Figure 5.8 Screenshot of Admin Home Screen	70
Figure 5.9 Screenshot of Admin Side Product Screen.....	71
Figure 5.10 Screenshot of Order Management Screen	72
Figure 5.11 Screenshot of Doctor Verification Management.....	73
Figure 5.12 Screenshot of Appointment Confirmation Screen.....	74
Figure 5.13 Screenshot of Doctor Profile.	75
Figure 5.14 Screenshot of Doctor chat screen	76
Figure 5.15 Screenshot of Doctor side Pet History Screen.....	77
Figure 5.16 Screenshot of Doctor Side Appointments Payment Screen	78

List of Tables

Table 1.1 Summary of Existing Veterinary Platforms.....	6
Table 3.1 Register / Login	20
Table 3.2 Search Veterinarians	21
Table 3.3 View Veterinarian Profile.....	22
Table 3.4 Book Appointment.....	23
Table 3.5 Online Payments	23
Table 3.6 Manage Pet Health Records.....	24
Table 3.7 Submit Review & Rating.....	25
Table 3.8 Search Medicines / Products.....	26
Table 3.9 View Medicine / Product Details.....	26
Table 3.10 Add Item to Cart	27
Table 3.11 Place Order	28
Table 3.12 Pay Online for Order.....	29
Table 4.1 Data Dictionary of Alhewan App	49
Table 5.1 External APIs for Alhewan App.....	60
Table 6.1 Manual Testing User Application.....	81
Table 6.2 Manual Testing Doctor Application	83
Table 6.3 Manual Testing Admin Panel	84
Table 6.4 System Testing for Alhewan App.....	85
Table 6.5 Unit Test Appointment Booking.....	86
Table 6.6 Unit Test Product Stock Management	87
Table 6.7 Unit Test Real Time Chat	87
Table 6.8 Functional Testing	88
Table 6.9 Integration Testing.....	90

CHAPTER 1

INTRODUCTION

1. Introduction

This chapter provides an overview of the Alhewan App, its purpose, and the motivation behind its development. It highlights the need for a digital veterinary system and outlines the problems with traditional methods. The introduction sets the foundation for understanding the system's objectives and core functionality. It also explains the technologies used and the structure of the overall project.

Veterinary healthcare plays a vital role in ensuring the well-being, productivity, and safety of animals. As pet ownership continues to rise and livestock management becomes increasingly important, the demand for structured veterinary services has grown significantly. However, in many regions, veterinary care still operates through manual and outdated practices such as physical appointment booking, handwritten medical records, and unverified professional information. These inefficiencies result in communication delays, lost medical histories, unreliable service quality, and limited access to credible veterinarians.

The Alhewan App is developed to address these challenges by providing a centralized, mobile-based solution for managing veterinary appointments, pet health records, secure online payments, and customer reviews. The system connects animal owners with verified veterinarians, allowing users to conveniently find veterinary professionals, schedule appointments, and maintain digital health records. Veterinarians can manage their profiles and appointment schedules, ensuring a smooth workflow. The admin oversees the system's operations, verifies veterinarians, and ensures secure user management.

Developed using Flutter for cross-platform mobile development and Firebase for backend functionalities, the Alhewan App offers real-time synchronization, secure authentication, responsive user interfaces, and reliable data storage. This chapter introduces the project, discusses its relevance, presents the background study, and outlines the methodology used in development.

The Alhewan App extends beyond a typical veterinary booking system by integrating multiple service layers into a single, interconnected platform. It ensures that every activity whether booking an appointment, documenting a pet's medical history, or conducting admin verification flows through a central ecosystem that maintains consistency and security. The introduction also highlights the system's focus on reducing the gap between veterinarians and animal owners by offering real-time updates, automated workflows, and structured data handling. Unlike traditional veterinary services, the Alhewan App makes every process traceable, organized, and accessible,

helping both pet owners and doctors manage animal healthcare with greater accuracy and convenience.

1.1 Project Brief

The Alhewan App is a mobile application designed to streamline veterinary workflows by offering animal owners and veterinarians a unified digital platform. It allows customers to register, create pet profiles, search for veterinarians, book appointments, view medical history, process payments, and submit reviews after services. Veterinarians can register, undergo admin verification, create professional profiles, set availability schedules, and manage their appointments through the system. Administrators handle veterinarian verification, user management, and overall monitoring of the platform. The integration of Firebase provides reliable backend services such as authentication, database storage, and cloud hosting.

The primary objective of the Alhewan App is to provide an automated veterinary care management solution that reduces manual effort, increases accessibility, and improves the overall user experience. The app supports pet owners, livestock managers, and veterinary professionals by enabling structured and organized service delivery. This project includes a multi-role environment where each user type interacts with the system in unique and complementary ways. Customers can manage multiple pet profiles, receive appointment confirmations instantly, and track treatment history in a unified dashboard. The veterinarian module offers tools to simplify daily tasks, including access to patient history, direct communication with clients, and clear appointment scheduling. Meanwhile, the admin module centralizes oversight, ensuring doctors are verified, product inventories are valid, and all transactions remain transparent. The strength of the project lies in its interconnected modules, where each update such as payment approval or appointment confirmation instantly reflects across all related sections due to real-time Firebase sync.

1.2 Relevance to the Course Module

This project directly aligns with the core principles and learning outcomes of software engineering, mobile application development, and system design methodologies studied in the academic curriculum. In this project, every development phase applies core BS(CS) concepts particularly in system modelling, backend architecture, and interface design. Requirement Engineering was directly applied to identify real problems faced by animal owners and veterinarians in Pakistan, shaping the SRS of Alhewan.

The design phase translated real-world veterinary workflows into UML models such as sequence diagrams and class diagrams, ensuring technical clarity. Implementation reflects core

programming knowledge in Flutter and Dart, while Firebase integration showcases real-time database concepts taught in cloud computing courses. The app's logic for appointment approval, payment verification, and chat demonstrates how theoretical algorithms were converted into practical features within the final application. Throughout the application's development, the following theoretical concepts from the course have been practically implemented:

1.2.1 Requirement Engineering

The Software Requirements Specification (SRS) provides detailed functional and nonfunctional requirements for the system. The Alhewan App applies requirement engineering concepts by translating real veterinary challenges into clear, actionable system requirements. The SRS developed for this project includes detailed functional needs such as pet profile management, appointment booking, payment verification, and doctor–patient communication. Non-functional requirements like usability, security, and performance were also documented to ensure the system works efficiently on all devices. Stakeholder interviews and observations helped refine requirements, making the SRS a direct reflection of real-world veterinary workflows. These requirements guided every development phase, ensuring the final system aligns with user needs and project objectives.

1.2.2 System Design and Modelling

Use case diagrams, sequence diagrams, architecture diagrams, and class diagrams from the Software Design Description (SDD) represent the structural and behavioural aspects of the project. The system design phase converted the SRS into visual and structural models that define how the Alhewan App operates internally. The use case diagram outlines the interactions between customers, veterinarians, and the admin, ensuring clear separation of responsibilities and permissions. Sequence diagrams were used to model the flow of critical processes such as booking, payment verification, and chat messaging. Class diagrams helped structure entities like User, Pet, Appointment, Review, and Payment within the Firebase backend. The architecture diagram provides a high-level overview of how Flutter, Firebase Authentication, Firestore, and Storage work together to support real-time data flow across all modules.

1.2.3 Software Development Life Cycle (SDLC)

The project follows an Agile and Iterative development model, as discussed in course topics. The project follows an Agile and Iterative SDLC to accommodate evolving features and continuous improvements. Each sprint delivered a functional module, such as authentication,

pet profiles, appointment scheduling, chat, and admin controls. Feedback from supervisors and test users guided modifications in later iterations, helping refine both usability and functionality. Agile was especially effective due to Firebase's real-time nature, allowing the team to test and adjust features quickly. The iterative cycle ensured the product remained aligned with user expectations and allowed the team to add complex components gradually without breaking existing functionality.

1.2.4 Database and Backend Concepts

Firebase Firestore and Cloud Functions demonstrate real-time data handling, authentication, and backend logic. The backend of the Alhewan App leverages Firebase Firestore to store user accounts, veterinarian data, pet profiles, appointments, reviews, and order details. Fire store's real-time synchronization was crucial for features such as chat, appointment updates, and payment status changes. Firebase Authentication handles secure login and role management for customers, veterinarians, and admins. Cloud Functions automate backend tasks such as updating appointment statuses, sending notifications, or validating data. The use of a NoSQL structure ensures scalability, allowing the system to support increasing numbers of users, pets, and appointments without performance issues.

1.2.5 User Interface Design

Application screens were designed following standard UI/UX guidelines taught in the course. The UI of the Alhewan App was built using Flutter's Material Design principles to maintain consistency, accessibility, and visual clarity across screens. Interfaces such as Login, Home, Category View, Appointment Calendar, and Doctor Details were structured to minimize user effort and reduce navigation complexity. Buttons, forms, icons, and colour schemes were chosen to create a calm, professional veterinary environment. The UI/UX flow encourages users to complete tasks such as booking or payment uploads with minimal steps. Visual hierarchy and clean layouts help users quickly identify important information, making the app intuitive for all age groups.

1.2.6 Testing and Quality Assurance

Test cases were designed and validated based on the SRS and SDD documents. The project reflects the academic foundation of software engineering while demonstrating the ability to develop a complete real-world application.

1.3 Background

The background of this project recognizes specific issues observed in local veterinary clinics: difficulty maintaining continuous pet records, limited communication between doctors and owners, and inconsistent appointment handling. Many clinics in Pakistan still rely on manual diaries, WhatsApp messages, and verbal confirmations, which often lead to lost records or scheduling errors.

The Alhewan system was designed after studying these real challenges especially the lack of verified veterinarian listings, confusion in payment confirmations, and missing medical histories for recurring patients. By centralizing these processes into a structured digital platform, the project directly addresses the gaps identified during initial surveys and stakeholder discussions. Many veterinary services lack a structured digital system for managing day-to-day operations. Common problems faced by pet owners and veterinarians include:

- Limited access to certified veterinarians.
- Manual appointment scheduling.
- Unorganized and scattered pet medical records.
- No automated recording of examination or treatment history.
- Lack of transparency in payment processes.
- No proper mechanism for customer feedback.
- Inability to track veterinarian service quality.

These challenges highlight the need for a comprehensive digital veterinary management system.

The Alhewan App solves these issues by digitizing appointment scheduling, pet health management, secure payment processing, and customer feedback collection. It offers a platform where users can maintain consistent and long-term health records of their pets and access veterinary services with transparency and ease.

1.4 Literature Review

Several veterinary platforms were studied to understand their strengths and limitations. The literature review also helped identify which features to include or avoid in Alhewan. For example, existing apps like Pet Desk and Vet Storia offer good scheduling features, but they lack doctor verification and detailed patient health management. This gap guided the team to prioritize modules such as veterinarian verification, multi-pet medical history tracking, and order/product handling. Similarly, research on cloud-based health systems informed the

decision to use Firebase for secure data storage and instant updates. The review of real-time communication apps influenced the inclusion of a chat system for doctor–customer interactions. Each insight from the literature directly informed the project’s design choices and module selection. While existing applications provide certain features such as appointment management or communication, none of them combine all essential veterinary service components into a single application.

1.4.1 Summary of Existing Veterinary Platform

The literature review table compares various existing veterinary platforms to identify their strengths and limitations relevant to digital pet care systems. It highlights how current applications focus on isolated features such as scheduling, reminders, or chat, but lack an integrated ecosystem combining medical history, doctor verification, payments, and product management. This comparative analysis helped the team identify critical gaps that the Alhewan App aims to address. The table also guided the selection of unique features, ensuring the project offers more comprehensive functionality than existing systems. Overall, the literature review table serves as a foundation for justifying the design decisions and scope of the Alhewan App. The Table 1.1 Summary of Existing Veterinary Platforms is explaining this summary.

Table 1.1 Summary of Existing Veterinary Platforms

Existing System	Strengths	Limitations
Pet Desk	Good for reminders and pet scheduling	Lacks structured medical record tracking
Vet Storia	Strong appointment booking features	No integrated pet health profiling
Pets App	Useful for chat-based interaction	Does not focus on reviews or vet verification
Otto. Vet	Clinic integrations	Limited features for independent vets
Next Inline	Queue and scheduling	No medical history management

The Alhewan App differentiates itself by combining user profiles, pet medical history, booking system, secure payments, admin verification, and customer reviews in one integrated platform.

1.5 Methodology and Software Life Cycle

Agile methodology combined with Iterative model was selected as illustrated in Figure 1.1, because it best matched the nature of the Alhewan Veterinary Management System. This project includes multiple interconnected modules and user roles, so a flexible approach was needed to manage changes, reduce risk, and ensure that each feature remains stable throughout development. The Agile approach was tailored to the needs of this project, allowing the team to handle complex modules such as real-time chat, appointment scheduling logic, and payment verification in separate sprints. Instead of developing the entire system in one long cycle, the team worked in short iterations where planning, development, and testing were performed repeatedly, resulting in continuous improvement.

Early iterations focused on building the foundation of the system, including authentication, account creation, role-based access (customer, veterinarian, and admin), and basic navigation flows. After the core structure was stable, later cycles were used to implement functional modules such as pet profile management, doctor availability and slot configuration, appointment booking, and payment proof uploading. This sprint-by-sprint structure made the project easier to manage because each sprint had clear goals, measurable deliverables, and a defined scope. It also ensured that the development team could focus on one module at a time while maintaining system consistency.

The real-time nature of Firebase required continuous testing and adjustments, which made Agile highly suitable. Because features such as chat messaging, appointment status updates, and admin verification workflows depend on live synchronization, the team frequently validated database rules, updated data models, and optimized queries to ensure smooth performance. Agile supported quick refinement and integration of these modules without disrupting the overall system. Sprint-based development also enabled the team to demonstrate progress to supervisors regularly, gather feedback, and implement improvements in the next sprint cycle. This feedback loop helped refine UI/UX decisions and confirmed that the system matched the expectations of end users. Each iteration produced a functional build that could be tested immediately on multiple devices, ensuring compatibility, responsiveness, and usability across different screen sizes. This approach minimized technical debt by identifying issues early, such as data validation problems, inconsistent UI behaviour, or role-based access conflicts, before they became larger and harder to fix. Furthermore, the iterative approach made it easier to refine data structures in Firebase by improving collection design, reducing redundancy, and ensuring accurate relationships between appointments, payments, and users.

UI layouts were also adjusted based on practical usage patterns observed during testing, which improved clarity and user experience.

As a result, the project evolved steadily with each cycle and sprint, producing a stable, scalable, and user-centred application. Agile with Iterative development ensured that the team could respond to change, maintain continuous quality through testing, and deliver modules in a controlled and organized manner. This combined methodology improved reliability, supported incremental feature expansion, and ensured that integration between the User App, Doctor App, and Admin Panel remained smooth throughout the development life cycle.

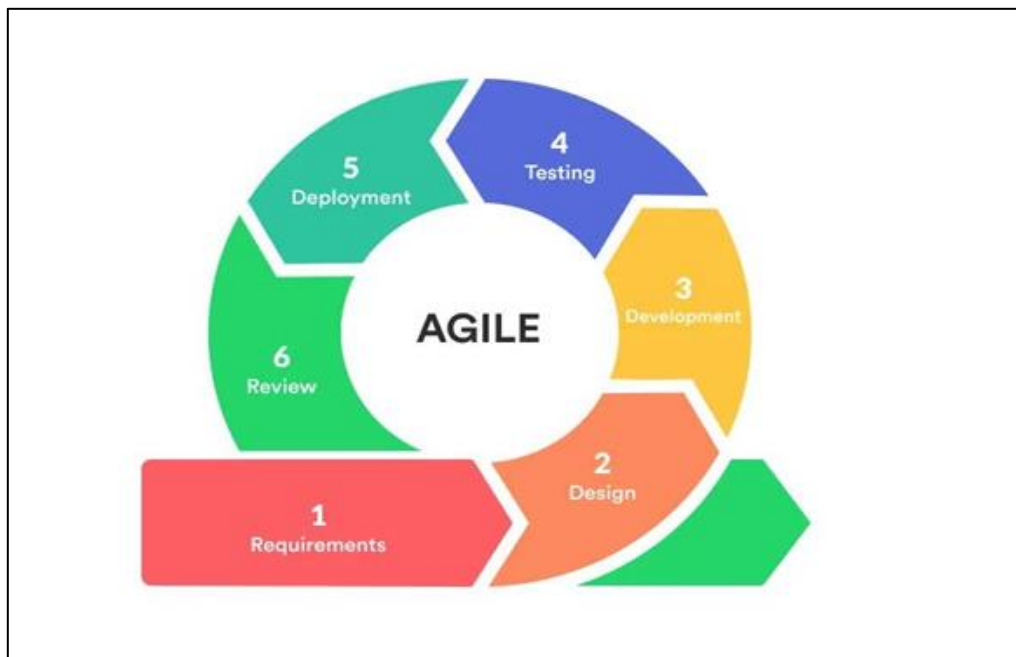


Figure 1.1 Life Cycle of Agile Methodology

This agile life cycle involves the following stages:

1.5.1 Requirement Analysis

Understanding stakeholder needs using SRS. In this stage, the team gathered detailed information from real users including pet owners, local veterinarians, and clinic staff to understand existing problems in veterinary service management. These insights were converted into precise functional and non-functional requirements documented in the SRS. Attention was given to practical needs such as screenshot-based payment verification, doctor verification workflows, and multi-pet health record management requirements that typical veterinary apps lacked. Each requirement was evaluated for feasibility within the project's timeline and technology stack.

1.5.2 Planning & Iteration Design

Organizing features into iterations (sprints). After defining requirements, the team divided the system into multiple sprints to ensure structured development. Each sprint focused on a single module for example, authentication, appointment booking, payment uploads, or chat. Prioritization was based on dependency order and user importance. Detailed sprint plans allowed the team to review progress weekly, adjust, and handle unexpected challenges like UI revisions or database restructuring without delaying the entire project

1.5.3 Design Phase

Use case diagrams, class diagrams, and UI designs are finalized. During this phase, all visual and structural models of the system were finalized. The use case and class diagrams ensured that user interactions and data relationships were clearly mapped before code development began. UI wireframes were created for major screens such as Login, Home, Doctor Details, and Appointment Calendar to establish a consistent flow. This design foundation prevented ambiguity during implementation and ensured all features aligned with user expectations and the SRS.

1.5.4 Implementation

Flutter frontend, Firebase backend, and module integration. This stage included the actual development of the Alhewan App using Flutter for the frontend and Firebase services for the backend. Real-time chat, screenshot upload, appointment scheduling, role-based dashboards, and admin controls were implemented gradually according to sprint timelines. The modular nature of Flutter allowed the team to develop reusable components, while Firebase's real-time capabilities ensured smooth synchronization across user types. Each module was integrated carefully to maintain system stability.

1.5.5 Testing

Module-by-module testing (login, appointment, payment, consultation). Every module underwent individual testing to ensure functionality, reliability, and usability. The team conducted scenario-based testing for key workflows like booking, payment approval, doctor verification, and chat messaging. Integration tests ensured that updates by one user (such as an admin approving a payment) reflected instantly for both doctors and customers. Bugs identified during testing phases were fixed before moving into deployment, ensuring a polished user experience.

1.5.6 Deployment

Release on Android & iOS platforms. Once the system was tested and stable, the app was deployed on Android and prepared for future iOS release. Firebase Hosting was used to deploy the admin panel for continuous access. Deployment involved configuring release builds, optimizing performance, and compressing resources to ensure smooth installation and runtime on mobile devices. The team conducted real-device testing to validate app behaviour outside the development environment.

1.5.7 Feedback & Review

New requirements added in next iteration. After deployment, feedback was collected from test users, veterinarians, and supervisors. This feedback uncovered valuable insights such as the need for clearer appointment status indicators, better UI spacing, and improved product categorization. New requirements emerging from this feedback were added to the next sprint cycle, ensuring continuous improvement and alignment with real-world usage.

1.6 Rationale Behind Methodology

The Agile approach allows the team to refine features based on testing feedback and supervisor suggestions without halting the entire development process. This ensured each module improved progressively. Core functionalities like login, appointment booking, and payment uploads were delivered early in development. This early availability helped identify usability issues and technical limitations sooner. Because each sprint focuses on a self-contained module, new features like doctor chat, product management, or payment history can be added without disturbing previously developed components.

Developing and testing the system incrementally minimized the risk of large-scale failure. Errors detected in small modules were easier to fix, preventing major delays in the overall project. Firebase's instant synchronization aligned perfectly with Agile, allowing the team to test real time updates such as chat messages, appointment approvals, and review submissions instantly across devices. Mobile apps often require frequent UI adjustments, rapid testing, and continuous optimization. Agile provides the flexibility needed to improve user interfaces, refine workflows, and adjust features quickly based on device compatibility and user interaction.

CHAPTER 2

PROBLEM DEFINITION

2. Problem Definition

This chapter identifies the challenges faced in current veterinary systems and the gaps in existing solutions. It explains the communication, appointment, and medical record issues affecting pet owners and veterinarians. The chapter defines clear project deliverables and requirements necessary to meet user needs. It establishes the justification for developing an automated and digital veterinary platform. It emphasizes how outdated communication, appointment, and record-keeping practices hinder both pet owners and veterinarians. The chapter further outlines precise project deliverables, development requirements, and the rationale for creating a digital veterinary platform that improves efficiency, accessibility, and the quality of care.

2.1 Problem Statement

Veterinary care services in many regions still operate through manual communication methods such as phone calls, physical clinic visits, handwritten medical histories, and scattered record-keeping practices. These outdated approaches frequently lead to delays, miscommunication, lost health records, and a lack of timely treatment for animals especially in emergencies.

Animal owners face significant challenges such as:

- Difficulty finding verified and trustworthy veterinarians.
- No centralized platform to book appointments.
- Manual tracking of vaccinations and medical history.
- Absence of reminders for upcoming treatments.
- Limited accessibility for livestock and rural areas.
- No structured platform for veterinarians to manage appointments.
- Poor coordination between users, veterinarians, and support staff.

Similarly, veterinarians struggle with:

- Unorganized appointment scheduling.
- Limited access to prior medical records.
- Difficulty managing payments and follow-ups.
- No system to issue secure digital prescriptions.

These issues create inefficiencies that directly impact the quality of veterinary care. Therefore, a comprehensive, automated, and user-friendly digital system is needed to modernize animal healthcare services.

2.2 Deliverables and Development Requirements

What resources are necessary to achieve these outcomes. Deliverables refer to tangible outputs or feature that directly address the identified problems, while development requirements include hardware, software, and human resources needed to implement the solution effectively. This section ensures that the project remains goal-oriented, with clearly defined objectives, and provides a roadmap for successful implementation.

2.2.1 Project Deliverables

Project deliverables include both functional and technical outputs. Functional deliverables focus on the user-facing aspects of the system, such as secure registration and login, veterinarian verification, pet profile management, appointment scheduling, online payments, and customer review mechanisms. Each functional module addresses specific gaps in the existing system. For instance, the veterinarian verification module ensures that only licensed professionals are allowed on the platform, enhancing trust and reliability for pet owners. The pet profile and health tracking module enables systematic recording of medical history, vaccinations, and treatment schedules, reducing the risk of missed preventive care.

The appointment booking module simplifies scheduling and supports both in-person and remote consultations, increasing accessibility, particularly for rural or underserved areas. Payment and billing automation reduces administrative workload and ensures financial transparency, while the customer review and rating system fosters accountability and continuous improvement among veterinarians. Technical deliverables complement these functional modules, providing the underlying infrastructure to support the system. A cross-platform mobile application developed with Flutter ensures accessibility on both Android and iOS devices. Firebase Firestore allows real-time data synchronization, enabling instant updates to appointments, records, and notifications. Firebase Authentication ensures secure login and protects sensitive pet health data.

The admin dashboard consolidates system management, including user monitoring, dispute resolution, and analytical reporting. Additionally, thorough documentation of the development process, testing results, and system usage ensures maintainability and facilitates future updates. Development requirements specify the resources necessary to realize these deliverables. Hardware resources include development workstations and mobile devices for testing, as well as a stable internet connection to facilitate cloud-based functionality. Software resources include the Flutter SDK, Dart programming environment, Android Studio, Firebase Console, and version control tools such as Git. Human resources include mobile app developers, backend

developers, UI/UX designers, QA testers, and project supervisors, each playing a crucial role in ensuring the quality, functionality, and user friendliness of the final product. By defining these requirements, the project ensures that the team is equipped with the necessary tools and expertise to develop an efficient and reliable digital veterinary platform.

2.2.1.1 Functional Deliverables

Functional Deliverables refer to the specific features or modules of a system that directly serve the end users and fulfill the core objectives of a project. In the context of a veterinary management platform, functional deliverables are the components that provide tangible, user-facing functionality, enabling pet owners and veterinarians to perform essential tasks efficiently. These deliverables define what the system will do to solve the identified problems, such as managing user accounts, booking appointments, tracking pet health, processing payments, and facilitating feedback. Functional deliverables are crucial because they translate the project's goals into actionable outcomes.

They ensure that each module or feature addresses a real-world need or gap in existing systems. For example, a pet health tracking module allows the system to store vaccination records and treatment histories, directly helping users monitor preventive care, while an appointment booking module enables structured scheduling, reducing errors and delays. Similarly, a payment module ensures smooth financial transactions, and a review module builds accountability and trust. By clearly defining functional deliverables, a project sets the standard for what users can expect the system to perform, guiding developers, designers, and stakeholders in building a product that is practical, user friendly, and aligned with the identified problems.

i. User Management Module

- Registration and Login system.
- Profile management for customers and veterinarians.

ii. Veterinarian Verification (Admin Panel)

- Admin reviews documents and verifies veterinarians.
- Admin manages disputes, payments, and system data.

iii. Pet Profile & Health Tracking Module

- Record medical history, vaccinations, treatment plans.
- Store pet details: species, breed, age, etc.

iv. Appointment Booking Module

- Search veterinarians.

- View available time slots.
- Book, reschedule, or cancel appointments.
- v. Payment & Billing Module**
 - Online payment integration.
 - Invoice generation.
 - Transaction handling and confirmation.
- vi. Customer Review & Rating Module**
 - Customers can rate veterinarians after completed appointments.
 - Users can submit written reviews about service quality and experience.
 - Average ratings and recent reviews are displayed on veterinarian profiles.
 - Admin can monitor, filter, or remove inappropriate reviews if needed.
- vii. Technical Deliverables**
 - Cross-platform mobile application (Flutter).
 - Firebase Firestore database.
 - Firebase Authentication.
 - Admin dashboard hosted on Firebase.
 - Testing documentation and complete project report.

2.2.2 Development Requirements

- i. Hardware Requirements**
 - Development laptop or PC.
 - Android/iOS device for testing.
 - Stable internet connection.
- ii. Software Requirements**
 - Flutter SDK
 - Android Studio
 - Dart programming environment
 - Firebase Console
- iii. Human Resource Requirements**
 - Mobile app developer
 - Backend developer
 - UI/UX designer
 - QA tester

- Project supervisor

2.3 Current System

Traditional veterinary systems rely on manual processes such as phone calls, messaging, and in-person visits for booking appointments. Pet medical histories are usually handwritten or stored in physical files, which are prone to loss or damage. There is limited support for structured financial transactions, and no systematic review mechanism for evaluating veterinarian services.

CHAPTER 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

This chapter describes what the system must do and how users will interact with it. It includes use-case diagrams, functional requirements, non-functional requirements, and workflow scenarios. Requirement analysis ensures that all system features are well-documented and aligned with user expectations. It acts as a blueprint for the later design and development phases. Requirement analysis is a critical phase in system development that establishes a clear understanding of what the Alhewan App must achieve and how users will interact with it.

This chapter goes beyond simply listing features; it evaluates user expectations, operational workflows, and system constraints to ensure a comprehensive blueprint for development. By analysing use cases, functional and non-functional requirements, and workflow scenarios, the development team can align system capabilities with the practical needs of customers, veterinarians, and administrators.

3.1 Use Case Diagram

The use case diagram of the Alhewan App highlights the relationships and interactions among the three main user types of customers, veterinarians, and administrators showing how each role contributes to the platform's overall functionality. Customers engage with the system to manage pet profiles, book appointments, make payments, and submit reviews, while veterinarians handle medical consultations, schedule management, and access pet health records.

3.1.1 Use Case Diagram of Alhewan App

The Figure 3.1 Use case diagram of Alhewan App there is illustration of the interactions between the three main users of the Alhewan App: Admin, Doctor, and Customer. Each user has distinct roles and functionalities within the system:

- i. The Admin manages core administrative tasks such as adding and managing products, verifying profiles, handling product deliveries, monitoring transactions, generating sales reports, managing appointments, and replying to queries.
- ii. The Doctor (veterinarian) handles medical-related tasks including managing appointments, writing prescriptions, reviewing past medical histories, receiving payments, and logging in and out of the system.
- iii. The Customer (animal owner) can browse and preview products, add items to their cart, review items, purchase products, pay online, book appointments, select doctors, and participate in virtual consultations. They can also log in and log out.

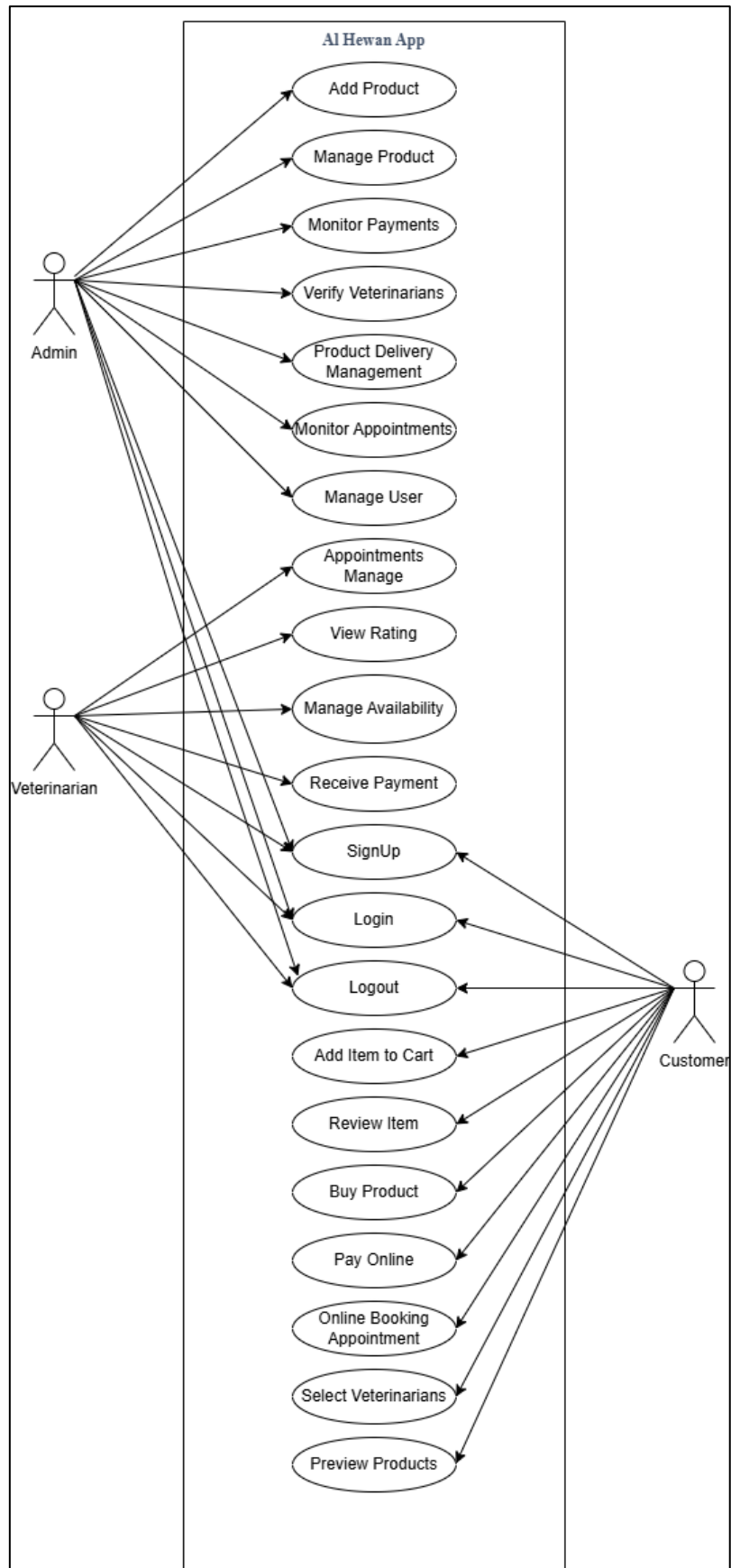


Figure 3.1 Use case diagram of Alhewan App

3.2 Use Case Descriptions

The Use Case Description outlines the key interactions between users and the system through specific scenarios. It details how customers, doctors, and admins perform essential tasks such as registering or logging in, searching and purchasing products, booking appointments, conducting virtual consultations, writing prescriptions, and handling payments. Each use case specifies the flow of actions, including normal steps, alternative options, and exceptions like errors or network failures. This structured approach helps ensure clarity in system functionality and guides development to meet user needs effectively.

3.2.1 Register / Login Use Case

This Table 3.1 Register / Login use case explains how customers, veterinarians, and admins can securely register or log in to the system. During registration, users provide their details, which the system validates before creating an account. For login, users enter their credentials, and the system verifies them to grant access to the appropriate dashboard. Incorrect or invalid credentials result in an error message. The process ensures that only authenticated users can access the system, maintaining security and smooth operation.

Table 3.1 Register / Login

Attribute	Description
Use Case ID	UC-01
Use Case Name	Register / Login
Actors	Customer, Veterinarian, Admin
Description	The user registers by providing required details or logs in with existing credentials. The system verifies the credentials and grants access. Incorrect credentials trigger an error message.
Preconditions	User must provide valid information.
Postconditions	User is authenticated and logged in.
Normal Flow	1. User opens login/register page. 2. Enters credentials. 3. System verifies credentials.

	4. Access granted.
Alternate Flow	1. User selects “Register”. 2. Enters details. 3. New account created.
Exceptions	Invalid credentials, unregistered account, network error.

3.2.2 Search Veterinarians Use Case

In this Table 3.2 Search Veterinarians the customer searches for veterinarians by entering keywords or applying filters such as specialization, rating, or availability. The system processes the query, retrieves the most relevant results, and displays a list of suitable veterinarians. If filters are used, the system narrows the results accordingly to help the customer easily find the best match.

Table 3.2 Search Veterinarians

Attribute	Description
Use Case ID	UC-02
Use Case Name	Search Veterinarians
Actors	Customer
Description	Customer searches for veterinarians using keywords or filters. The system retrieves matching results.
Preconditions	Customer must be logged in.
Postconditions	Relevant veterinarian list is displayed.
Normal Flow	1. Customer enters search text. 2. System retrieves results. 3. List displayed.
Alternate Flow	Customer applies filters (specialization, rating, availability).
Exceptions	No results found, network issue.

3.2.3 View Veterinarian Profile Use Case

In this Table 3.3 View Veterinarian Profile the customer can view the full veterinarian profile before booking an appointment. This includes important details such as qualifications, years of experience, consultation fee, availability, reviews, and ratings. After the customer selects a veterinarian, the system loads and displays all relevant information so the customer can make an informed decision.

Table 3.3 View Veterinarian Profile

Attribute	Description
Use Case ID	UC-03
Use Case Name	View Veterinarian Profile
Actors	Customer
Description	Customer views detailed veterinarian profile including qualifications, experience, availability, reviews, and ratings.
Preconditions	Customer selects a veterinarian.
Postconditions	Veterinarian details are shown.
Normal Flow	1. Customer selects vet. 2. System loads details. 3. Information displayed.
Exceptions	Profile unavailable, network error.

3.2.4 Book Appointment Use Case

In this Table 3.4 Book Appointment this use case describes how a customer books an appointment with a veterinarian by selecting an available date and time. After choosing a veterinarian, the customer is shown all available time slots. Once the customer selects a suitable time, the system confirms the appointment and records the booking. This process ensures the customer can easily schedule a consultation without any difficulty.

Table 3.4 Book Appointment

Attribute	Description
Use Case ID	UC-04
Use Case Name	Book Appointment
Actors	Customer
Description	Customer schedules an appointment by choosing a veterinarian and selecting an available time slot.
Preconditions	Customer logged in; available slots exist.
Postconditions	Appointment is booked.
Normal Flow	1. Customer selects vet. 2. System shows available slots. 3. Customer selects time. 4. Appointment confirmed.
Alternate Flow	Customer reschedules before confirmation.
Exceptions	No available slots, network failure.

3.2.5 Online Payments Use Case

In this Table 3.5 Online Payments, This use case explains how customers complete online payments for their booked appointments. After scheduling an appointment, the customer selects a preferred payment method (such as Easypaisa, Jazz Cash, or bank transfer). The system securely processes the transaction and confirms the payment. Once the payment is successful, the system records it and updates the appointment status accordingly.

Table 3.5 Online Payments

Attribute	Description
Use Case ID	UC-05
Use Case	Online Payments

Name	
Actors	Customer, Veterinarian
Description	Customer completes the payment for the booked appointment. System processes the transaction securely.
Preconditions	Appointment booked.
Postconditions	Payment recorded and confirmed.
Normal Flow	1. Customer selects payment method. 2. System processes payment. 3. Confirmation displayed.
Alternate Flow	Customer chooses a different payment method.
Exceptions	Payment failure, insufficient balance, network issue.

3.2.6 Manage Pet Health Records Use Case

This Table 3.6 Manage Pet Health Records explains how customers manage the health records of their pets. The customer can view important details such as vaccination history, age, medical treatments, and reminders. After selecting a pet profile, the system retrieves and displays all stored information. Customers can also update or edit these records whenever needed, ensuring that their pet's medical history remains accurate and up to date.

Table 3.6 Manage Pet Health Records

Attribute	Description
Use Case ID	UC-06
Use Case Name	Manage Pet Health Records
Actors	Customer
Description	Customer views, updates, and maintains pet health details such as vaccinations, age, medical history, and reminders.
Preconditions	Pet profile exists.

Postconditions	Updated or viewed pet data displayed.
Normal Flow	1. Customer selects pet. 2. System retrieves details. 3. Records displayed.
Alternate Flow	Customer edits pet information.
Exceptions	Data missing, network error.

3.2.7 Submit Review and Rating Use Case

This use case Table 3.7 Submit Review & Rating explains how customers provide feedback after their appointment. Once a consultation is completed, the customer can submit a star rating along with optional written comments. The system then stores this feedback so future customers can view the veterinarian's overall rating and reviews.

Table 3.7 Submit Review & Rating

Attribute	Description
Use Case ID	UC-07
Use Case Name	Submit Review & Rating
Actors	Customer
Description	Customer submits feedback through star rating and optional comments after appointment.
Preconditions	Appointment completed.
Postconditions	Review stored in system.
Normal Flow	1. Customer opens appointment history. 2. Customer enters rating/comment. 3. System saves review.
Exceptions	Network failure, submission error.

3.2.8 Search Medicines / Products Use Case

This use case Table 3.8 Search Medicines / Products describes how customers search for pet medicines and products from the home page. Customers can enter keywords or use category filters to find relevant items. The system processes the search query and displays matching results, helping users quickly locate the products they need.

Table 3.8 Search Medicines / Products

Attribute	Description
Use Case ID	UC-08
Use Case Name	Search Medicines / Products
Actors	Customer
Description	Customer searches for pet medicines/products using keywords or categories. System retrieves matching results.
Preconditions	Customer logged in.
Postconditions	Search results displayed.
Normal Flow	1. Enter search query. 2. System retrieves items. 3. Results displayed.
Alternate Flow	Apply filters (category, availability, price, rating).
Exceptions	No items found, network failure.

3.2.9 View Medicine / Product Details

This use case Table 3.9 View Medicine / Product Details explains how customers view complete details of a selected medicine or product. After clicking on an item, the system loads and displays information such as usage instructions, price, photos, dosage, and customer reviews. This helps customers make informed purchasing decisions.

Table 3.9 View Medicine / Product Details

Attribute	Description
-----------	-------------

Use Case ID	UC-09
Use Case Name	View Medicine / Product Details
Actors	Customer
Description	Customer views full details including usage instructions, price, photos, and customer reviews.
Preconditions	Customer selected a product.
Postconditions	Item details displayed.
Normal Flow	1. Customer selects product. 2. System loads details. 3. Details displayed.
Alternate Flow	Customer adds item to favourites.
Exceptions	Product data unavailable.

3.2.10 Add Item to Cart Use Case

This use case Table 3.10 Add Item to Cart describes how customers add a selected product to their shopping cart. When the customer clicks “Add to Cart,” the system checks item availability and then adds it to the cart. Cart totals are updated immediately, and users can also adjust the quantity before checking out.

Table 3.10 Add Item to Cart

Attribute	Description
Use Case ID	UC-10
Use Case Name	Add Item to Cart
Actors	Customer
Description	Customer adds a selected item to the cart. System checks availability and updates cart totals.

Preconditions	Customer selected a product.
Postconditions	Product added to cart.
Normal Flow	1. Click “Add to Cart”. 2. System checks inventory. 3. Item added to cart.
Alternate Flow	Customer adjusts quantity.
Exceptions	Out of stock, network issue.

3.2.11 Place Order Use Case

This use case Table 3.11 Place Order explains the process of placing an order after reviewing all cart items. The customer reviews the cart, proceeds to checkout, confirms delivery details, and places the order. The system verifies product stock and then creates the order successfully.

Table 3.11 Place Order

Attribute	Description
Use Case ID	UC-11
Use Case Name	Place Order
Actors	Customer
Description	Customer reviews cart, proceeds to checkout, confirms delivery details, and places order.
Preconditions	Items must be in the cart.
Postconditions	Order is created.
Normal Flow	1. Customer reviews cart. 2. Clicks “Checkout”. 3. System verifies stock. 4. Order created.
Alternate Flow	Customer modifies cart before placing order.

Exceptions	Stock unavailable, system error.
-------------------	----------------------------------

3.2.12 Pay Online for Order

This use case Table 3.12 Pay Online for Order explains how customers make online payments for their orders. After placing an order, the customer selects a payment method and completes the transaction. The system verifies the payment and updates the order status to “Paid,” ensuring the order is ready for processing and delivery.

Table 3.12 Pay Online for Order

Attribute	Description
Use Case ID	UC-12
Use Case Name	Pay Online (Order Payment)
Actors	Customer
Description	Customer pays for their order using an online payment method. The system verifies the transaction and updates order status.
Preconditions	Order must exist.
Postconditions	Payment confirmed and order marked as paid.
Normal Flow	1. Select payment method. 2. System processes payment. 3. System confirms transaction.
Alternate Flow	Customer changes payment method before confirmation.
Exceptions	Network errors, insufficient balance, payment failure.

3.3 Functional Requirements

Functional requirements are the specific tasks, features, or actions that a system must be able to perform to meet the needs of its users. They define what the system should do and describe the behaviour and operations required to achieve its objectives. In the context of a veterinary management system, functional requirements include the ability to register and log in users, manage pet profiles, schedule appointments, process payments, send notifications, and store

medical records. These requirements focus on the operational capabilities of the system and directly determine the functionality available to users. For example, the system must allow users to create accounts, veterinarians to update treatment records, pet owners to book or cancel appointments, and generate invoices for services. Functional requirements are crucial for ensuring that the system delivers the services it was designed to provide and meets user expectations.

Modules such as the customer, veterinarian, and admin modules, as well as appointment booking, pet health tracking, payment, and review systems, define the core capabilities of the Alhewan App. Each module is designed to serve a specific purpose while interacting seamlessly with others. For example, pet health records are linked to appointments, enabling veterinarians to access relevant medical history during consultations, while the review module feeds back into the system to maintain service quality. Functional requirements serve as the foundation for building the system, specifying what tasks the app must perform and ensuring operational completeness.

3.3.1 Customer Module

i. **Description:**

Enables pet owners to register, log in, and manage their user profile. Customers can add detailed information about their pets, including breed, age, vaccination history, and medical records.

ii. **Requirements:**

User registration, authentication, profile management, pet profile creation, pet record updates.

3.3.2 Veterinarian Module

i. **Description:**

Allows veterinarians to register on the platform, complete their professional profile, and get verified by the admin. Veterinarians can manage their availability, view appointments, and access relevant pet health records.

ii. **Requirements:**

Vet registration, verification status, profile completion, schedule management, appointment viewing.

3.3.3 Admin Module

i. **Description:**

The admin oversees the entire platform by verifying veterinarians, managing user accounts, monitoring appointments, and ensuring smooth system operations.

ii. **Requirements:**

Vet verification, user management, appointment monitoring, system reporting.

3.3.4 Appointment Booking Module

i. **Description:**

Allows customers to book appointments with veterinarians based on available time slots. Customers can also reschedule or cancel existing appointments.

ii. **Requirements:**

Slot selection, appointment creation, rescheduling, cancellation, appointment tracking.

3.3.5 Pet Health Tracking Module

i. **Description:**

Enables customers to store, view, and manage their pets' medical records. The system can display vaccination history and upcoming healthcare reminders.

ii. **Requirements:**

Medical record storage, vaccination history, reminder alerts for vaccines/check-ups.

3.3.6 Payment & Billing Module

i. **Description:**

Provides secure online payment processing for appointments. Generates invoices and maintains transaction records for both customers and veterinarians.

ii. **Requirements:**

Payment gateway integration, invoice generation, transaction history storage.

3.3.7 Review & Feedback Module (Customer Only)

i. **Description:**

Allows customers to submit ratings and written feedback for veterinarians after their appointments. These reviews help improve service quality and guide other users.

ii. **Requirements:**

Rating submission, written feedback, display of reviews on veterinarian profile.

3.4 Non-Functional Requirements

Non-functional requirements complement functional requirements by defining the system's quality attributes, including usability, performance, security, portability, and reliability. Usability ensures that all users can navigate the app intuitively, complete tasks efficiently, and access information with minimal effort. Performance requirements guarantee that the system responds quickly, processes transactions smoothly, and retrieves data without lag. Security measures protect sensitive user and pet health information from unauthorized access, maintaining confidentiality and trust. Portability ensures the app functions consistently across Android, iOS, and web-based admin interfaces, while reliability focuses on high availability, automated backups, and failover mechanisms to prevent disruptions. Together, these non-functional requirements ensure that the Alhewan App not only functions correctly but also deliver a secure, efficient, and user-friendly experience under real-world conditions. The requirement analysis phase also identifies dependencies and potential limitations, helping the development team plan for scalable and maintainable architecture. By integrating both functional and non-functional requirements into a coherent framework, the Alhewan App can address the challenges faced by pet owners and veterinarians, streamline operational workflows, and provide a robust digital platform for veterinary healthcare management. This comprehensive approach ensures that the system meets current needs while remaining adaptable for future enhancements, such as advanced payment integrations, telemedicine features, or automated reminders for pet care.

3.4.1 Usability

The system must offer an intuitive and easy-to-navigate interface. Customers should be able to register, book appointments, and manage pet records with minimal steps. Both customers and veterinarians should have clear dashboards for managing their tasks efficiently.

3.4.2 Performance

The system should deliver fast performance with quick database queries and minimal loading time. Appointment booking, pet record retrieval, and payment processing must be responsive. All pages should load smoothly without lag.

3.4.3 Security

Security is ensured using Firebase Authentication, encrypted data storage, and role-based access control. All pet health information, user data, and payment details must remain confidential and protected from unauthorized access.

3.4.4 Portability

The application should work seamlessly on **Android and iOS** devices. Additionally, the admin should have access to a **web-based panel**, ensuring consistent performance across multiple platforms.

3.4.5 Reliability

The system should provide **99.9% uptime**, cloud-based data storage, and automated backups to prevent data loss. Failover mechanisms should ensure continuous service availability without interruptions.

CHAPTER 4

DESIGN AND ARCHITECTURE

4. Design and Architecture

This chapter explains the internal structure of the system and how different components interact. It includes system architecture diagrams, process flows, sequence diagrams, and class diagrams. The design ensures modularity, scalability, and efficient data management across all system modules. It provides a technical roadmap for developers to implement the system accurately. The design and architecture of the Alhewan App provide a comprehensive overview of how the system is structured internally and how its components communicate with each other to deliver a seamless experience.

This chapter goes beyond visual diagrams to explain the relationships between modules, data flow, and the organization of core functionalities. The architecture emphasizes modularity, ensuring that each component, such as user management, pet records, appointment booking, and payments, operates independently yet integrates smoothly with other parts of the system. This approach allows the application to be scalable, maintainable, and adaptable for future feature expansions, such as adding telemedicine or automated reminders.

4.1 System Architecture

The Figure 4.1 System Architecture of Alhewan App divides the application into two primary modules: the User & Pet Management Module and the Services & Transactions Module. The User & Pet Management Module is responsible for user registration, login, authentication, and the management of pet profiles, including medical history and vaccination records. This module ensures that sensitive user and pet information is securely stored, retrieved efficiently, and linked accurately to related transactions or appointments. The Services & Transactions Module manages interactions such as searching for veterinarians, booking appointments, purchasing products, processing payments, and submitting reviews.

It ensures that operational workflows between customers, veterinarians, and administrators are executed correctly and consistently. Data flow in the Alhewan App is designed to maintain efficiency and reliability. Each module communicates with dedicated databases: the User Database stores login credentials and personal details, the Pet & Health Records Database maintains medical histories and vaccination logs, the Appointment Database tracks booking details, the Product/Medicine Database manages inventory and product information, the Payment Database records financial transactions, and the Review Database stores customer feedback.

This separation of data storage allows for optimized queries, easier maintenance, and secure handling of confidential information. The architecture also incorporates real-time updates and

notifications to keep users informed about appointment statuses, payments, and other critical actions, enhancing user engagement and system responsiveness. Furthermore, the design ensures a clear separation of concerns through layered architecture. The presentation layer handles the mobile interface built with Flutter, providing a responsive and intuitive experience for both Android and iOS users. The business logic layer processes user requests, validates data, and applies system rules, while the data access layer manages secure communication with Firestore and Firebase Storage. This separation enables developers to implement, test, and maintain individual components without affecting unrelated modules, resulting in a robust and flexible system. The architecture also includes mechanisms for error handling, authentication, and role-based access control. Users are authenticated securely using Firebase Authentication, and role-based permissions ensure that customers, veterinarians, and administrators can only access features relevant to their responsibilities. This protects sensitive data, prevents unauthorized operations, and maintains overall system integrity. Additionally, backup and failover strategies are integrated to enhance reliability, ensuring that the system remains operational even in case of hardware or network issues. By establishing a structured architecture, the Alhewan App provides a technical roadmap that guides developers through implementation, integration, and future upgrades. It ensures that each module functions efficiently while interacting seamlessly with other components, delivering a comprehensive, secure, and user-friendly platform for managing veterinary services, pet health, and related transactions. This approach not only meets current user needs but also provides a strong foundation for extending system capabilities in the future. This figure illustrates the high-level architecture of the Alhewan App. The system is divided into two main modules: the User & Pet Management Module, which handles user registration, login, and pet health records, and the Services & Transactions Module, which manages veterinarian search, appointment booking, product purchases, online payments, and reviews. These modules interact with multiple logical databases such as the User Database, Pet and Health Records Database, Appointment Database, Product and Medicine Database, Payment Database, and Review Database. Each database is responsible for storing and managing a specific category of system data to ensure proper organization and data integrity. The User Database securely maintains authentication credentials and profile information for customers, veterinarians, and administrators. Pet and Health Records are stored separately to allow accurate tracking of medical history and treatment details. Appointment data is managed to support scheduling, status updates, and history tracking. Product and Medicine databases facilitate inventory management and order processing.

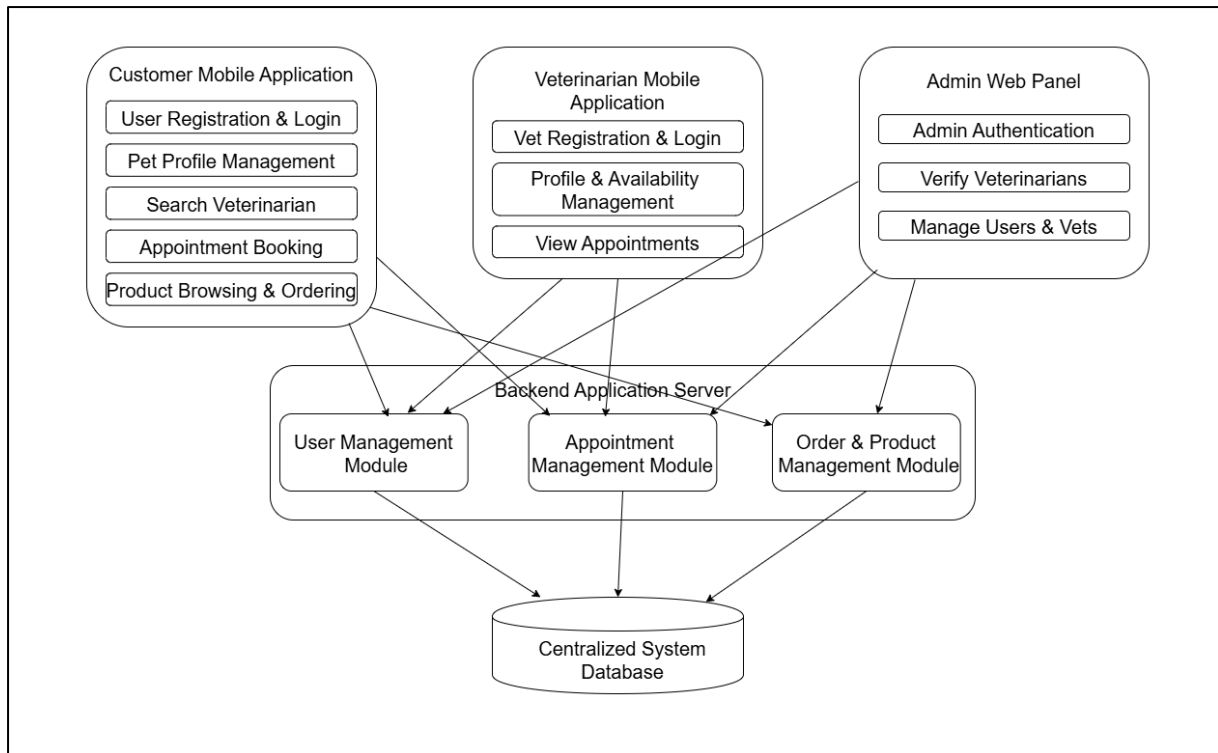


Figure 4.1 System Architecture of Alhewan App

4.2 Process Flow

This Figure 4.2 Process Flow of Alhewan App demonstrating how different user roles, including Customer, Veterinarian, and Admin, interact with the system from application launch to task completion. The process begins when the user opens the mobile application, which immediately displays the login screen. If the user is not registered, they can navigate to the registration screen where personal information such as name, email, phone number, and password is entered. During registration, the user selects the account type as either Customer or Veterinarian. Once the account is created, the system checks the account type. If the user is a veterinarian, the system assigns the status “Pending Verification,” sends a verification request to the administrator, and displays a message instructing the veterinarian to await approval. Customers, however, receive a “Registration Successful” message and are redirected to the login screen for authentication.

After registration, the user logs into the system by entering an email and password. The backend server validates the credentials against stored records in the database. If authentication fails, the system displays a login error message and denies access to the platform. If authentication is successful, the system retrieves the user role from the database and redirects the user to the appropriate dashboard. This role-based redirection ensures that users only gain access to the features that are relevant to their responsibilities. This mechanism is a key security

feature that prevents unauthorized access to sensitive functionalities such as vet verification or system monitoring.

For the Customer workflow, the system displays the Customer Dashboard after successful login. The customer can retrieve or edit their personal profile and manage pet profiles, including entering details such as pet name, breed, age, vaccination history, and general health records. The customer can then search for veterinarians based on specialization, availability, and ratings. By viewing vet profiles, customers can make informed decisions regarding appointment booking. Once a suitable vet is selected, the customer chooses a date and time slot and confirms the appointment, which is then stored in the system database.

Following appointment booking, the customer can move to the shopping module where medicines and pet products are displayed. Customers can browse products, view item details, add products to the cart, update quantities, or remove items. After reviewing the cart, the customer places the order and proceeds to online payment. If the payment is successful, the system confirms the order, generates a receipt, and stores the transaction details in the database. In case of failure, the system prompts the user to retry. Finally, customers can submit reviews and ratings for veterinarians and purchased products, contributing to system transparency and service quality improvement.

For the Veterinarian workflow, once the vet logs in, the system first retrieves the vet profile and checks the verification status. If the veterinarian is not verified, the system displays the message “Not verified yet” and redirects the vet to the verification status page. After admin approval, the veterinarian can access full system features. These include managing availability schedules, adjusting working hours, viewing assigned appointments, and opening appointment details. Veterinarians can also update the status of appointments as completed or cancelled and access pet health information for better treatment planning.

The veterinarian module also allows vets to review feedback and ratings submitted by customers. This feature enables veterinarians to monitor patient satisfaction and improve their services. All actions performed by veterinarians are logged in the system database, ensuring accountability and performance tracking. This workflow ensures that vets are fully integrated into the system while maintaining strict control over who is allowed to offer professional services. For the Admin workflow, after successful login, the Admin Panel is displayed. The administrator has system-wide privileges and is responsible for verifying, approving, or rejecting veterinarian accounts. The admin can manage vet accounts, including activating, deactivating, or updating profiles. Additionally, the admin can manage medicines and products,

monitor appointments to track platform activity, monitor orders and payments for auditing purposes, and view user reviews and ratings to ensure service quality.

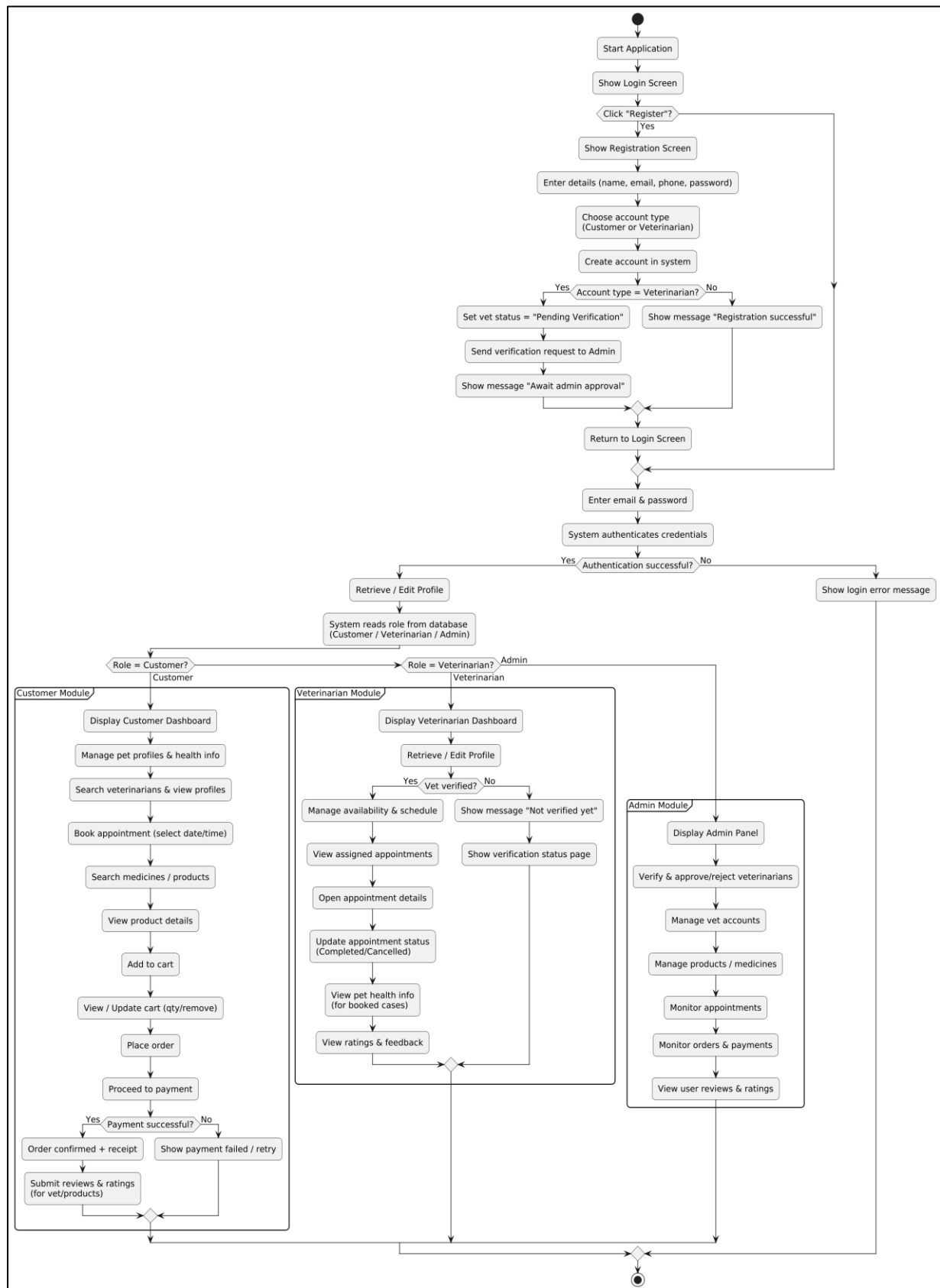


Figure 4.2 Process Flow of Alhewan App

4.3 Design Models

4.3.1 Sequence Diagram

The Figure 4.3 Sequence Diagram of Alhewan App represents the interaction between the user and the main modules of the Alhewan App over time. Starting with user authentication, the user then communicates with the Pet Management Module to view or update pet profiles, followed by the Search Veterinarian and Appointment Modules to find a vet and book an appointment. Next, the user interacts with the Product/Medicine and Cart & Order Modules to search products, add them to the cart, and place an order. The Payment Module processes the transaction, and finally the Review & Rating Module stores the user's feedback, completing the end-to-end workflow.

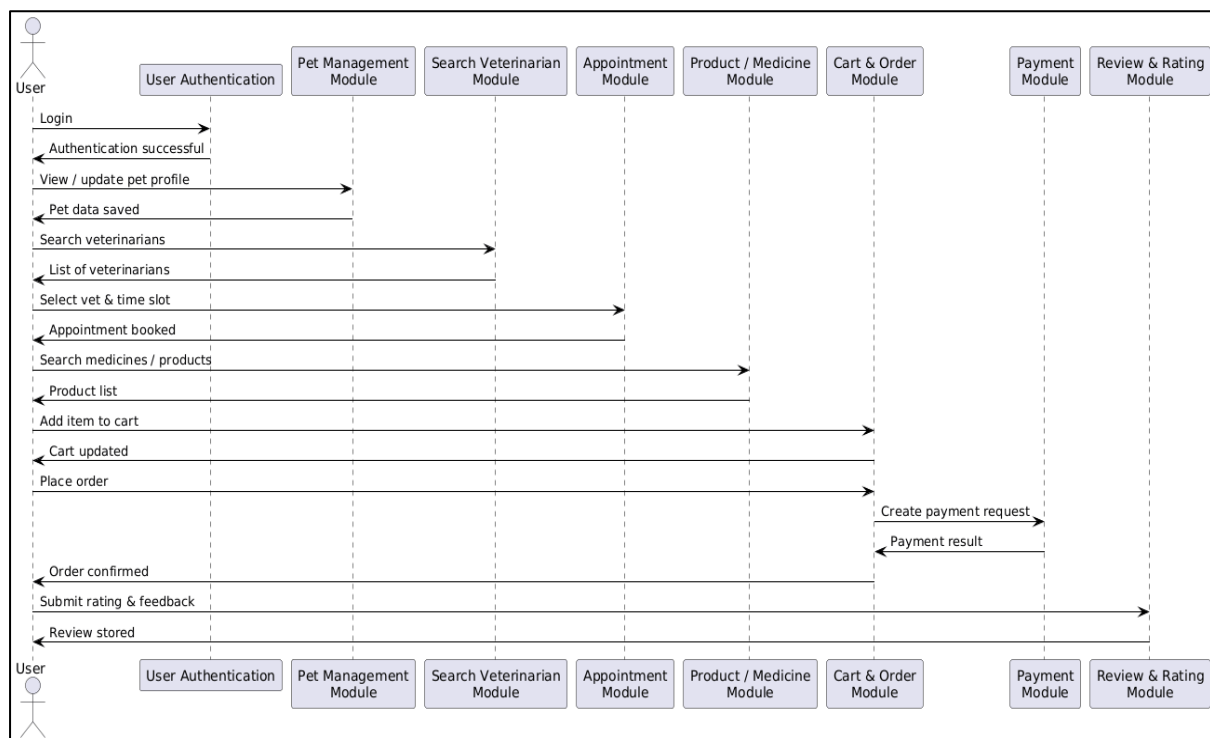


Figure 4.3 Sequence Diagram of Alhewan App

4.3.1.1 User Authentication Module Sequence Diagram

The Figure 4.4 User Authentication Module Sequence Diagram describes the complete login workflow of the Alhewan app from the user interface to the backend database. The process starts when the user opens the application and reaches the Material View Login Screen, where they enter their email and password and press the login button. The login screen sends the entered credentials to the User Authentication Module, which acts as the main controller for validation. The module first performs basic checks such as empty fields and input format before forwarding the request to the User Database. The database verifies whether the provided email

exists and whether the password matches the stored hash/record. Once validation is completed, the database returns a response to the authentication module indicating success or failure along with relevant user data. If authentication is successful, the module generates or retrieves a session token and sends a success response back to the login screen. The app then navigates the user to the Home/Profile Screen to continue normal usage. After navigation, the Home/Profile screen requests the authentication module to fetch the full profile details using the token or userId. The module queries the database again to retrieve profile information such as name, contact details, and user preferences. The database returns the profile data, and the module delivers it to the Home/Profile screen to display to the user. This ensures the dashboard is loaded with updated and personalized information. If authentication fails at any point, the module returns an error response (e.g., “Invalid credentials”) to the login screen. The login screen then displays the error message and prevents access to protected screens.

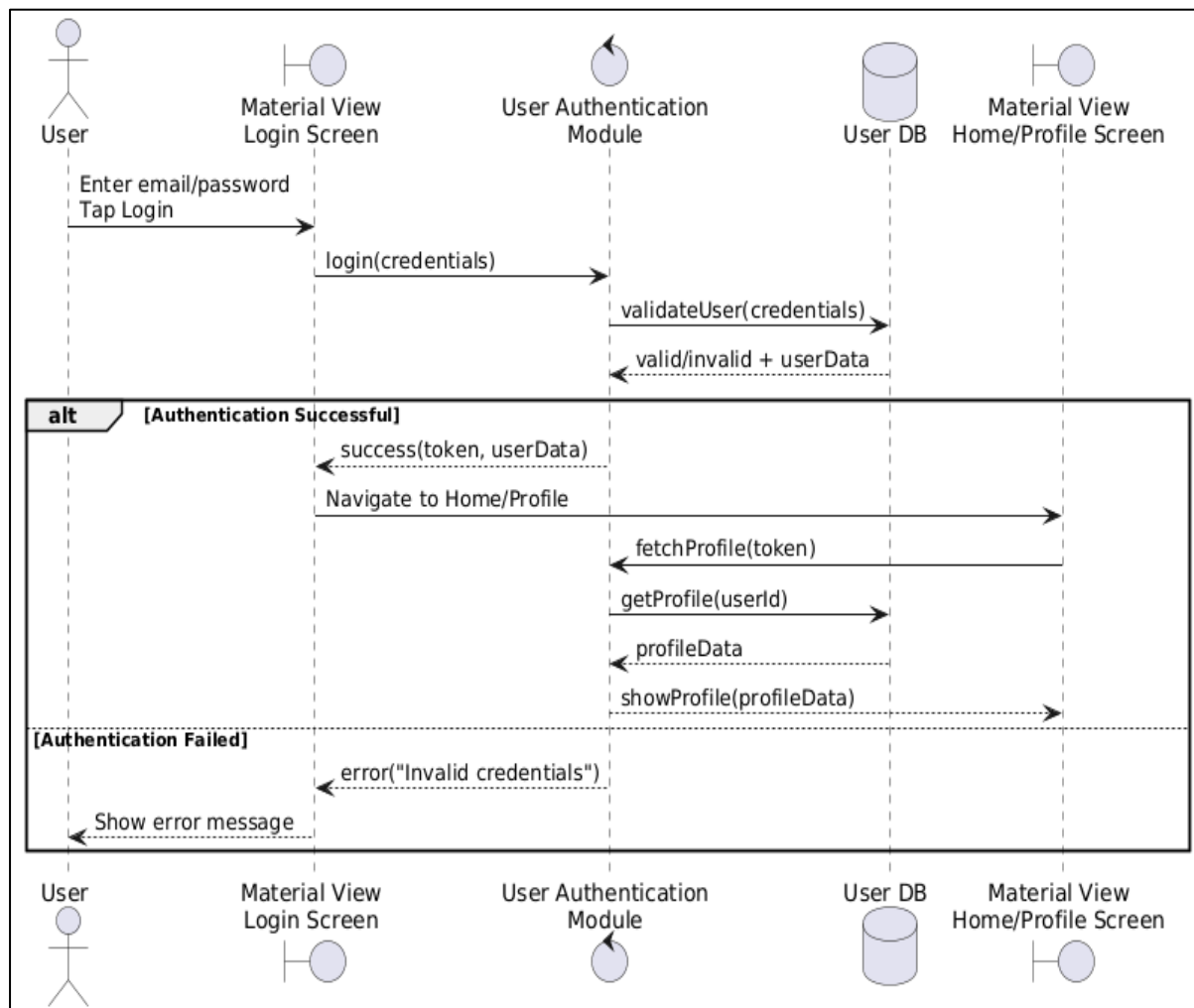


Figure 4.4 User Authentication Module Sequence Diagram

4.3.1.2 Appointment Booking Sequence Diagram

The Figure 4.5 Appointment Booking Sequence Diagram explains how a customer finds a suitable vet and successfully books an appointment through the Alhewan app. The process begins on the Material View Search Vet Screen, where the user enters search filters such as city/location, specialization, availability, and rating, then taps the search button. The Search Vet Screen sends the filter request to the Search Vet Module, which processes the criteria and queries the Vet Database to fetch matching veterinarians. The database returns a vet list containing key details (name, clinic, specialty, rating, and availability), and the module sends this list back to the screen for display. The user reviews the list, selects a veterinarian, and the system transitions to the Material View Appointment Screen. On the Appointment Screen, the user selects a preferred date and time slot based on the vet's available schedule. The Appointment Screen forwards the selected slot information to the Appointment Module to verify availability. The Appointment Module checks the Schedule Database to confirm whether the requested slot is still free and not already reserved by another customer. If the schedule database returns "available," the appointment module proceeds to create an appointment record in the Appointment Database with customerId, petId, vetId, and slot details. After saving the appointment, the module locks the same slot in the schedule database to prevent double booking. The system processes the appointment request by validating the selected date and time against the Schedule Database. If the requested slot is available, the Appointment Module creates a new appointment record and generates a confirmation response. This confirmation is then returned to the Appointment Screen, where the user interface displays a successful booking message to the user. The system also updates the appointment status to ensure accurate scheduling records. In case the Schedule Database indicates that the selected slot is not available, the Appointment Module immediately stops the booking process. No appointment record is created in the database under this condition. Instead, an error response is sent back to the Appointment Screen. The user interface displays a clear and informative message such as "Slot not available" to ensure that the user understands the reason for the unsuccessful booking attempt. The system then prompts the user to select an alternative date or time from the available schedule options. This immediate feedback helps guide the user toward making a valid selection without confusion. By performing real-time validation before confirming the appointment, the system effectively prevents scheduling conflicts. It ensures that no overlapping or duplicate appointments are created in the database.

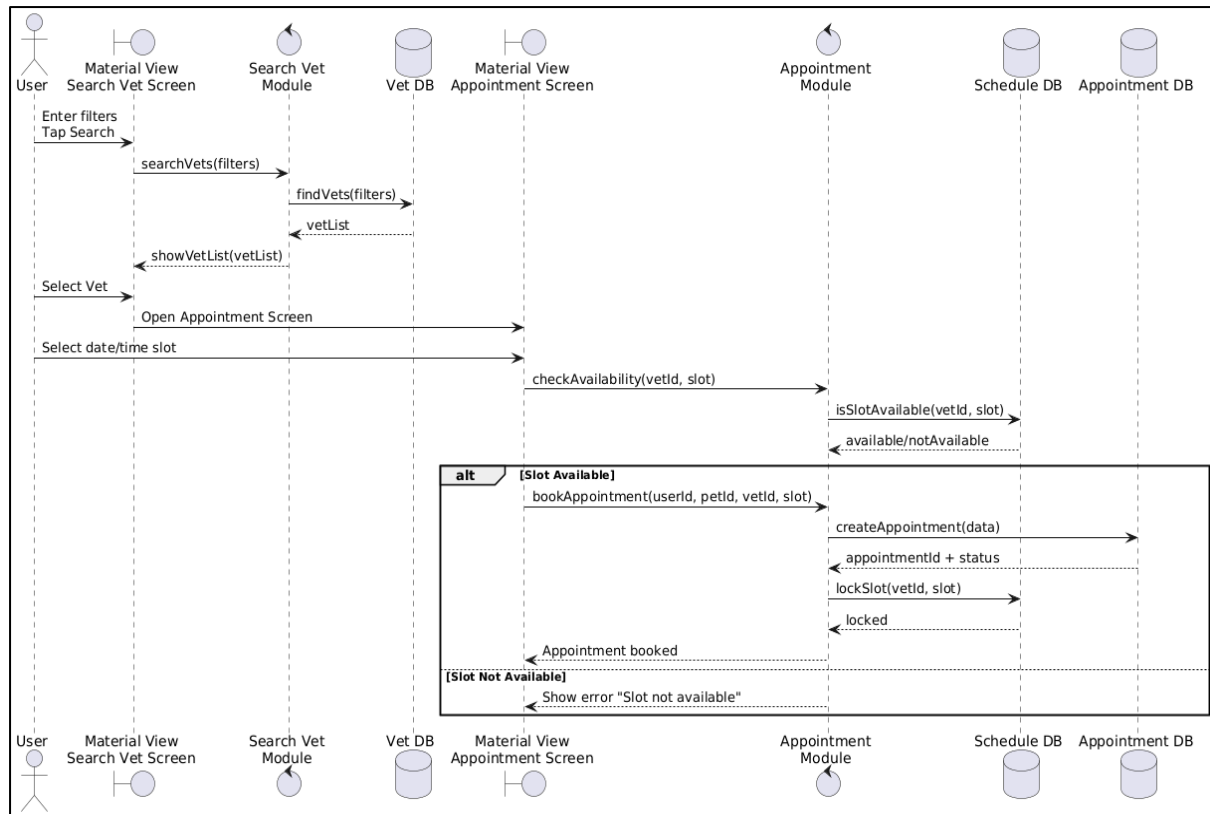


Figure 4.5 Appointment Booking Sequence Diagram

4.3.1.4 Vet Approval / Verification Sequence Diagram

The Figure 4.6 Vet Approval / Verification Sequence Diagram explains how the administrator validates and controls veterinarian onboarding in the Alhewan system. The workflow starts when the admin logs in and opens the Material View Admin Dashboard, where a section for “Pending Vet Applications” is available. The admin selects this option, and the dashboard sends a request to the Admin Management Module to fetch all vets whose verification status is pending. The admin module queries the Vet Database to retrieve the list of pending applicants along with basic profile information such as name, license/registration number, clinic details, and submitted documents. The database returns the pending vet list, and the admin module forwards it to the dashboard for display. The admin then selects a specific veterinarian application to review detailed documents, credentials, and verification proofs. After reviewing, the admin chooses either Approve or Reject, and the dashboard sends this decision to the admin management module with the vetId. If the admin approves the vet, the module updates the veterinarian’s status in the Vet Database to “Approved”, enabling the vet account to become active in the system. The system then triggers the Notification Service to send an approval message (email/SMS/app notification) informing the vet that they can now access vet features and receive bookings. If the admin rejects the vet, the module updates the status to “Rejected”,

and the vet remains inactive or blocked from providing services. A rejection notification is then sent, usually including a reason or instruction to resubmit documents if required. This sequence ensures strong access control, prevents unverified practitioners from operating, improves user trust, and maintains the quality and safety standards of the platform.

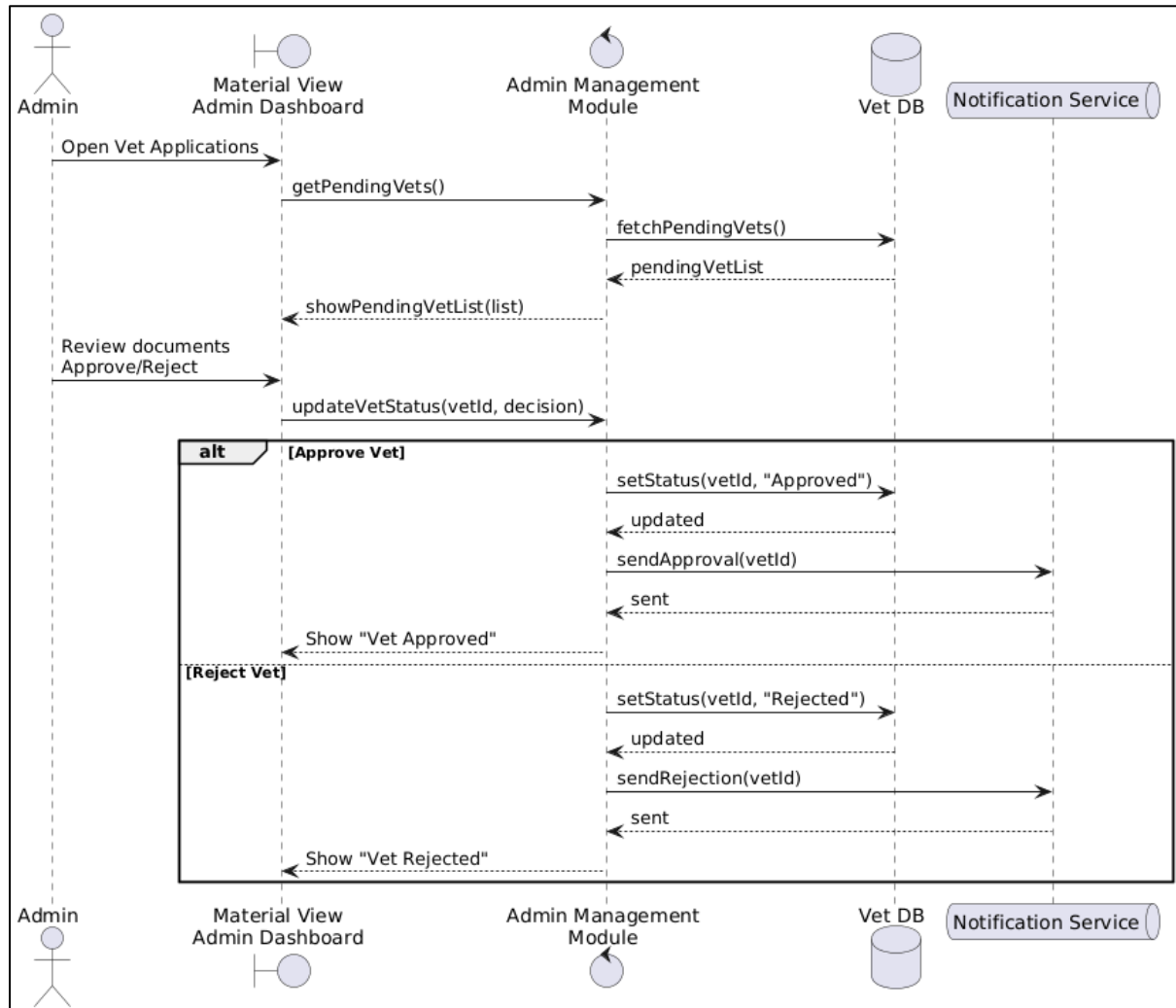


Figure 4.6 Vet Approval / Verification Sequence Diagram

4.3.1.5 Veterinarian Module Sequence Diagram

This Figure 4.7 Veterinarian Module Sequence Diagram describes how a veterinarian manages appointments inside the Alhewan system without involving prescriptions or medication handling. The process starts when the veterinarian opens the Material View Vet Dashboard and selects the option to view today's appointments. The dashboard sends a request to the Veterinarian Module with the vetId and the selected date to retrieve the schedule. The module queries the Appointment Database and fetches all appointments assigned to that veterinarian for the given date. The database returns an appointment list containing key information such

as appointment time, customer name, pet name, and current status. The veterinarian module then forwards this list to the dashboard, where it is displayed in an organized appointment view. After reviewing the schedule, the veterinarian selects a specific appointment to open its details. The dashboard sends a request to the veterinarian module to retrieve complete appointment details using the appointment. The module requests detailed records from the appointment database, including booking information, pet-related notes, and any previous appointment history if available. Once the database returns the detailed data, the module sends it back to the dashboard so the veterinarian can review the appointment context before action. When the veterinarian finishes the consultation or wants to update the appointment lifecycle, they change the appointment status (for example, Completed or Cancelled) from the dashboard. The dashboard sends the status update request to the veterinarian module, which validates the request and updates the appointment record in the appointment database. After the database confirms that the appointment has been updated, the module returns a success response to the dashboard. Finally, the dashboard displays a confirmation message showing that the appointment status has been updated successfully. This sequence ensures accurate appointment tracking, supports daily schedule management, and maintains reliable appointment history within the system.

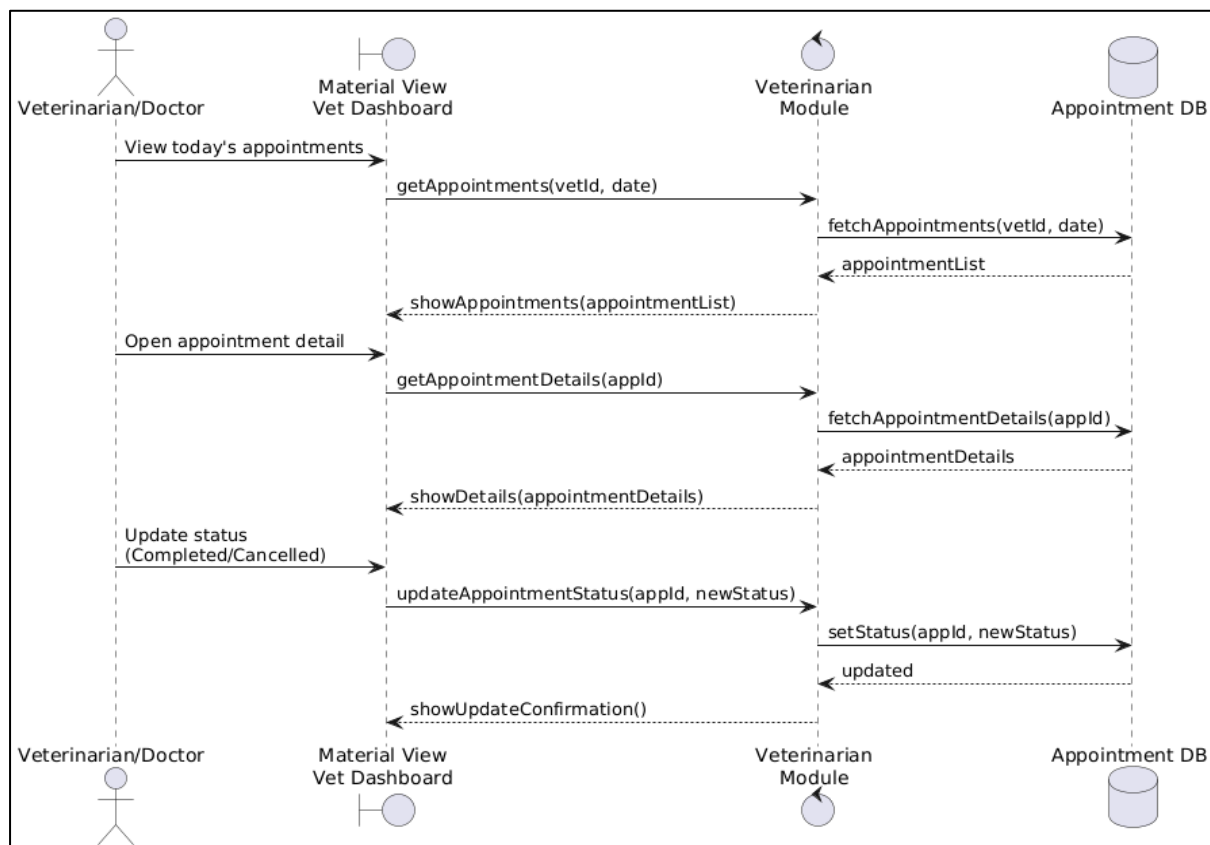


Figure 4.7 Veterinarian Module Sequence Diagram

4.4 Class Diagram

The Figure 4.8 represents the static structure of the system and illustrates how the core entities interact with each other through defined relationships, attributes, and operations. At the center of the design is the User class, which acts as a base class for three specialized roles: Customer, Veterinarian, and Admin. This inheritance structure enables role-based access control while avoiding data redundancy by allowing all common user attributes such as name, email, password, and phone number to be maintained in a single parent class. Each role extends the base user with role-specific attributes, such as address for customers, specialization and verification status for veterinarians, and staff identifiers for administrators. The diagram clearly defines system responsibilities and ensures scalability by allowing future roles to be added without major restructuring. Overall, this layered user model establishes a strong foundation for authentication, authorization, and personalized system behaviour.

The diagram further details how customers manage pets and appointments within the system. Each User in the customer role can own multiple Pet records, where each pet maintains detailed information including species, breed, age, medical history, and vaccination records. The association between Pet and Vaccination History ensures that long-term health data is properly recorded and traceable. Customers can book multiple Appointment instances for their pets, and each appointment is linked to a specific veterinarian, pet, date, and status. This design prevents scheduling conflicts and supports appointment lifecycle management such as booking, cancellation, rescheduling, and completion. By clearly defining multiplicities and constraints, the diagram ensures data consistency and supports real-world veterinary clinic workflows.

Veterinarian-related operations are modelled to ensure quality control and trust within the platform. The Veterinarian class includes professional details such as specialization, years of experience, clinic location, and verification status. To regulate onboarding, the Verification Request class enables veterinarians to submit credentials for administrative review. Each request maintains timestamps and status values, allowing the system to track pending, approved, or rejected applications. The Admin class is responsible for reviewing these requests and updating veterinarian status accordingly. This verification mechanism ensures that only approved professionals can provide services, thereby enhancing system credibility and user safety. Additionally, veterinarians can receive multiple Review entries, allowing performance evaluation based on customer feedback.

E-commerce functionality is represented through a well-defined set of shopping and transaction-related classes. Customers interact with the Product class to browse medicines, food, and accessories, which are categorized and managed for stock availability. The Cart and

CartItem classes work together to manage selected products, quantities, and pricing in a structured manner. Once checkout is initiated, an Order is generated and associated with a corresponding Payment record. The Payment class supports multiple payment methods and tracks transaction status, invoice generation, and refund handling. This modular design allows the system to manage both appointment-based payments and product purchases efficiently, while maintaining transactional integrity and traceability.

Finally, system communication and feedback are handled through the Notification and Review classes. Notifications are generated for important events such as appointment confirmations, payment updates, and veterinarian approval decisions, ensuring timely communication with users. Reviews allow customers to rate veterinarians using a standardized scale and provide textual feedback, which supports transparency and informed decision-making for future users. The relationships defined between users, reviews, and notifications emphasize user engagement and service quality monitoring. Overall, the class diagram presents a comprehensive and cohesive system architecture that aligns with the functional requirements of the Alhewan application, ensuring maintainability, extensibility, and real-world applicability.

In addition to structural clarity, the class diagram also emphasizes data integrity and system maintainability through well-defined associations and controlled dependencies. Each class encapsulates its own data and behaviour, ensuring that responsibilities are clearly separated and aligned with object-oriented design principles. For example, appointment and payment handling are isolated within their respective classes, reducing coupling between medical and commercial functionalities. The use of status attributes in core entities such as Appointment, Order, Payment, and Verification Request enables the system to manage state transitions in a structured and controlled manner. Each entity progresses through well-defined states, allowing the system to accurately track its lifecycle from initiation to completion. This approach simplifies error handling by clearly identifying incomplete, failed, or cancelled operations. It also supports auditing by maintaining a transparent history of changes and actions performed within the system. Status tracking improves reporting by allowing administrators to generate meaningful insights based on current and past system activities. Furthermore, this design supports future enhancements such as analytics, automated notifications, and reminder services. Policy enforcement rules can also be implemented more easily using status-based conditions. Overall, the architecture promotes consistency, reliability, and scalability. The diagram reflects a robust and extensible design capable of handling increasing user demand. This ensures long-term system stability while allowing seamless functional expansion.

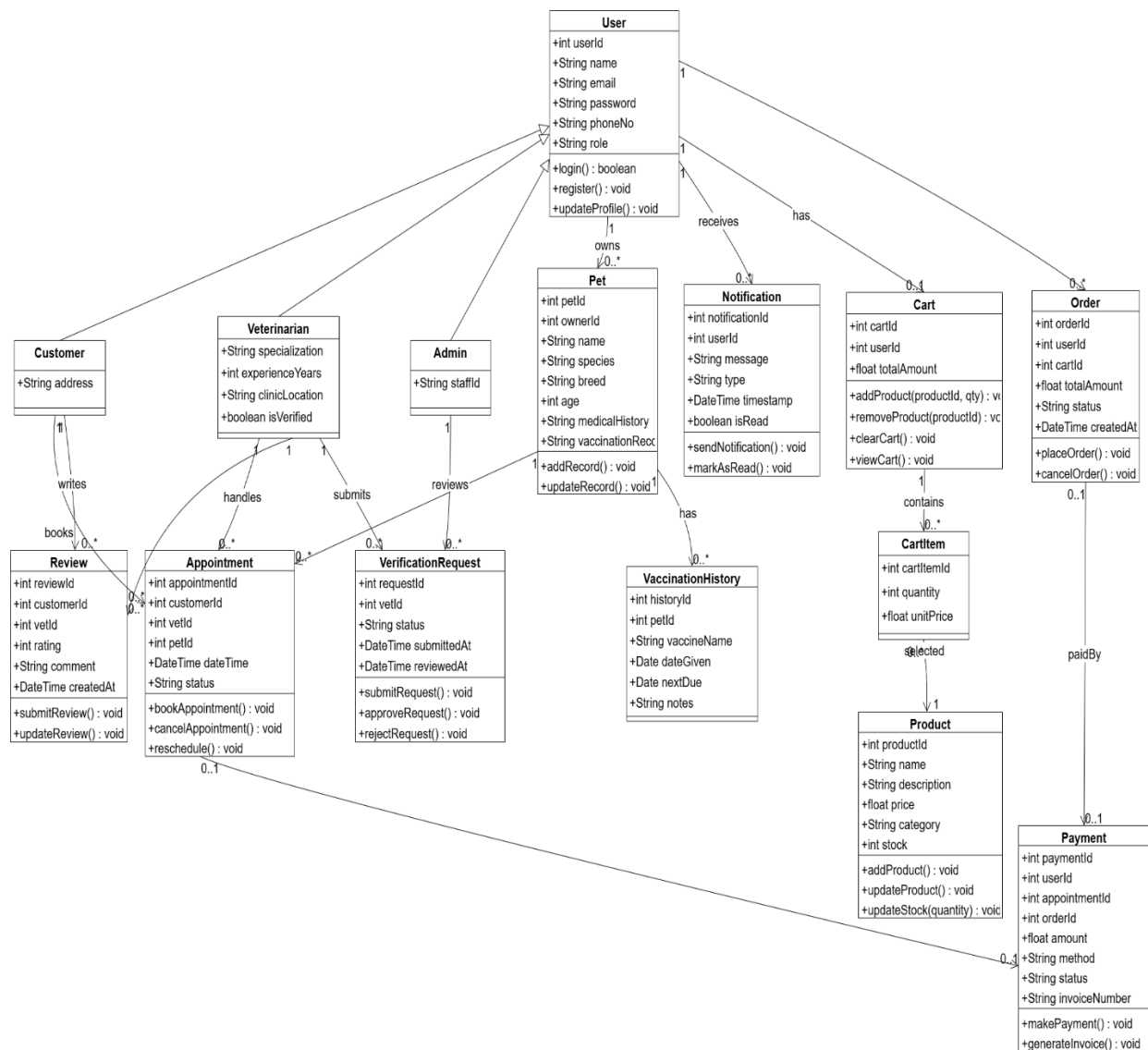


Figure 4.8 Class Diagram of Alhewan App

4.5 Data Design

The Data Design section defines the structure and purpose of all key data elements used in the Alhewan app. Each field in the data dictionary plays a critical role in ensuring accurate data storage, retrieval, and system-wide consistency. Unique identifiers such as User ID, Vet ID, Pet ID, Appointment ID, and Payment ID help maintain relational integrity between user accounts, veterinary profiles, pet records, appointment schedules, and financial transactions. Fields like Review ID, Rating, and Comment support the feedback system by storing user-generated reviews and ratings that enhance veterinarian credibility and improve service quality. By organizing and standardizing this information, the data dictionary provides a clear blueprint for backend operations, database structuring, and seamless integration across the user app, doctor app, and admin panel. This structured data foundation ensures that the system remains

scalable, reliable, and capable of handling real-time updates effectively. The Table 4.1 Data Dictionary of Alhewan App is describing it:

Table 4.1 Data Dictionary of Alhewan App

Field	Description
User ID	Unique user identifier
Vet ID	Veterinarian unique identifier
Pet ID	Pet profile ID
Appointment ID	Booking reference
Payment ID	Transaction ID
Review ID	Customer review record
Rating	Numeric score 1–5
Comment	Written review

CHAPTER 5

IMPLEMENTATION

5. Implementation

In this phase, we outline the implementation of the Alhewan App, a mobile-based veterinary appointment and pet health management system. The application focuses on practical and essential features such as user authentication, pet profile creation, appointment booking, manual mobile-wallet payments, and admin verification. The implementation phase of the Alhewan App not only focuses on the development of individual modules but also emphasizes the integration and interaction of these components to ensure a cohesive and seamless experience for users. Beyond the functional modules, the system architecture is designed to support scalability, data security, and real-time updates.

The app follows a modular structure, where each module, such as pet profile management, appointment booking, payment handling, and review submission, communicates efficiently with the Firebase backend. This ensures that changes in one module, such as updating pet health records, automatically reflect in related modules like upcoming appointments or notifications without manual intervention. The app also incorporates real-time features and notifications, which are critical for enhancing user engagement and ensuring timely communication between pet owners and veterinarians.

For example, updates on appointment status, payment confirmations, or chat messages are delivered instantly using Firebase Cloud Messaging and Firestore real-time listeners. This reduces delays, minimizes miscommunication, and improves overall responsiveness. Additionally, the system incorporates role-based access control, which ensures that customers, veterinarians, and administrators can only access features relevant to their roles, maintaining data privacy and system security. The project architecture, illustrated in Figure 4.2, provides a comprehensive view of how the mobile application, backend services, and admin dashboard interact. It shows the flow of data from the user interface to the Firebase database, storage, and messaging services. For instance, when a user books an appointment, the request is sent to Firestore, where the admin can review payment screenshots and approve or reject the appointment. Once approved, the status is updated in real time on both the user's and veterinarian's dashboards.

The architecture highlights a clear separation of concerns, with dedicated layers for presentation, business logic, and data management, ensuring maintainability and scalability for future enhancements. Furthermore, the system design incorporates offline support and error handling mechanisms, allowing users to continue certain activities even with intermittent internet connectivity. Forms and data submissions are validated before being sent to the

backend, reducing errors and ensuring data integrity. In case of failed authentication, invalid uploads, or cancelled actions, the system provides meaningful feedback to the user, guiding them on corrective steps.

This ensures a smooth user experience and reduces frustration, particularly for users with limited technical proficiency. In addition to these features, the app's user interface emphasizes simplicity and accessibility. Screens are logically organized, and navigation is designed to minimize user effort. Features such as categorized product browsing, detailed veterinarian profiles, and clear status indicators for appointments make the application intuitive. The admin panel 47 complements this by providing a centralized view of users, veterinarians, orders, and appointments, enabling efficient management and monitoring of the system. Overall, the implementation phase demonstrates a careful balance between functional completeness, technical efficiency, and user-centered design. By integrating real-time data handling, secure authentication, role-based access, and a responsive interface, the Alhewan App ensures that both pet owners and veterinarians can manage veterinary care effectively, accurately, and conveniently.

i. Database Setup

- Firebase Firestore is used as the backend database to store user profiles, veterinarian records, pet information, appointments, payment screenshots, and reviews.
- Collections are structured for efficient data retrieval and real-time synchronization.
- Role-based access control is implemented to protect confidential data and restrict admin-level operations.

ii. User Interface Development

- The UI is developed using Flutter to ensure a consistent and responsive experience on both Android and iOS devices.
- Key screens include Login/Register, Dashboard, Pet Profile, Appointment Booking, Payment Upload, and Appointment Status.
- The design is kept simple and user-friendly as the system does not include audio/video consultations or complex modules.

iii. Authentication System

- Firebase Authentication is utilized to manage user login and registration.
- The system supports three user roles: Customer, Veterinarian, and Admin.
- Verified veterinarians are added and approved by the admin before they appear in the appointment booking list.

iv. Appointment Booking System

- Users can select a veterinarian, choose a preferred date and time, and submit an appointment request.
- Once an appointment request is submitted, it is marked as Pending Payment.
- There is no real-time video or audio consultation system in the app.

v. Manual Payment & Screenshot Upload

- Instead of an integrated payment gateway, users make payments manually via EasyPaisa or Jazz Cash using the veterinarian's mobile wallet number.
- After transferring the payment, users upload the payment screenshot directly in the app.

The admin reviews the uploaded screenshot and either *approves* or *rejects* the appointment request.

- Upon approval, the appointment status automatically changes to Confirmed.

vi. Pet Profile Management

- Users can add details about their pets including name, category, age, breed, and basic medical history.
- The pet information is linked with appointments for better record management.
- Users may update their pet information at any time.

vii. Review System

- After a completed appointment, users can provide ratings and written feedback for the veterinarian.
- Reviews are securely stored and displayed on the veterinarian's profile for transparency.

viii. System Testing & Optimization

- The system is tested thoroughly for appointment flow accuracy, payment verification functionality, and status transitions.
- Issues related to form validation, image upload, and page navigation are addressed.
- Performance optimizations are applied to enhance responsiveness and usability.

5.1 Algorithm

5.1.1 User Authentication Algorithm

Step 1: User enters email and password.

Step 2: System verifies credentials via Firebase Authentication.

Step 3: User role (Customer, Veterinarian, Admin) is retrieved.

Step 4: User is redirected to the corresponding dashboard.

- **User Authentication (Pseudo Code)**

```
PROCEDURE UserLogin(email, password)

  IF email is empty OR password is empty THEN

    SHOW "Please enter email and password"

    RETURN

  ENDIF

  TRY

    authUser = FirebaseAuth.SignIn(email, password)

    userId = authUser.uid

    userDoc = Firestore.Users.Get(userId)

    role = userDoc.role // Customer | Veterinarian | Admin

    IF role == "Customer" THEN

      NAVIGATE(CustomerDashboard)

    ELSE IF role == "Veterinarian" THEN

      NAVIGATE(VetDashboard)

    ELSE IF role == "Admin" THEN

      NAVIGATE(AdminDashboard)

    ELSE

      SHOW "Role not found"

    ENDIF

  CATCH error

    SHOW "Authentication failed"

  ENDTRY
```

```
ENDPROCEDURE
```

5.1.2 Pet Profile Algorithm

Step 1: User enters pet details in the form.

Step 2: System validates and stores the pet data in Firestore.

Step 3: User can update or modify the pet profile as needed.

Step 4: Pet profile connects automatically to appointment records.

- **Pet Profile (Pseudo Code)**

```
PROCEDURE SavePetProfile(userId, petData)
```

```
  IF petData.name is empty OR petData.age is invalid THEN
```

```
    SHOW "Invalid pet data"
```

```
    RETURN
```

```
  ENDIF
```

```
  petId = GenerateUniqueId()
```

```
  petData.userId = userId
```

```
  petData.createdAt = CurrentTime()
```

```
  Firestore.Pets.Set(petId, petData)
```

```
  SHOW "Pet profile saved successfully"
```

```
ENDPROCEDURE
```

```
PROCEDURE UpdatePetProfile(petId, updatedData)
```

```
  IF Firestore.Pets.Exists(petId) == FALSE THEN
```

```
    SHOW "Pet not found"
```

```
    RETURN
```

```
  ENDIF
```

```
  Firestore.Pets.Update(petId, updatedData)
```

```
  SHOW "Pet profile updated"
```

ENDPROCEDURE

5.1.3 Appointment Booking Algorithm

Step 1: User selects a veterinarian.

Step 2: User chooses the appointment date and time.

Step 3: System creates a new appointment with the status “Pending Payment.”

Step 4: User is redirected to the payment instructions page.

- **Appointment Booking (Pseudo Code)**

```
PROCEDURE BookAppointment(userId, petId, vetId, dateTime)
```

```
  IF dateTime is null THEN
```

```
    SHOW "Select date and time"
```

```
    RETURN
```

```
  ENDIF
```

```
  IF SlotAvailable(vetId, dateTime) == FALSE THEN
```

```
    SHOW "Slot not available"
```

```
    RETURN
```

```
  ENDIF
```

```
  appointmentId = GenerateUniqueId()
```

```
  appointment = {
```

```
    id: appointmentId,
```

```
    userId: userId,
```

```
    petId: petId,
```

```
    vetId: vetId,
```

```
    dateTime: dateTime,
```

```
    status: "Pending Payment",
```

```
    createdAt: CurrentTime()
```

```

}

Firestore.Appointments.Set(appointmentId, appointment)

NAVIGATE(PaymentInstructionsPage, appointmentId)

ENDPROCEDURE

```

5.1.4 Manual Payment & Screenshot Upload Algorithm

Step 1: User pays manually through Easy Paisa/Jazz Cash using the veterinarian's mobile number.

Step 2: User uploads the payment screenshot in the app.

Step 3: Admin reviews the screenshot for validity.

Step 4: If valid, admin approves the appointment → status = "Confirmed."

Step 5: If invalid, admin rejects the request → status = "Rejected."

- **Manual Payment & Screenshot Upload (Pseudo Code)**

```

PROCEDURE UploadPaymentScreenshot(appointmentId, imageFile)

  IF imageFile is null THEN

    SHOW "Upload payment screenshot"

    RETURN

  ENDIF

  path = "payments/" + appointmentId + ".jpg"

  url = FirebaseStorage.Upload(path, imageFile)

  Firestore.Appointments.Update(appointmentId, {

    paymentProofUrl: url,

    status: "Waiting for Admin Approval"

  })

  SHOW "Payment proof submitted"

ENDPROCEDURE

PROCEDURE AdminVerifyPayment(adminId, appointmentId, decision)

```

```

IF decision == "Approve" THEN

    Firestore.Appointments.Update(appointmentId, { status: "Confirmed" })

ELSE IF decision == "Reject" THEN

    Firestore.Appointments.Update(appointmentId, { status: "Rejected" })

ENDIF

SHOW "Verification updated"

ENDPROCEDURE

```

5.1.5 Review Submission Algorithm

Step 1: User selects a completed appointment.

Step 2: User enters rating (1–5) and a written review.

Step 3: System saves the review in Firestore.

Step 4: Review becomes visible on the veterinarian's profile

- **Review Submission (Pseudo Code)**

```

PROCEDURE SubmitReview(userId, appointmentId, rating, comment)

    appointment = Firestore.Appointments.Get(appointmentId)

    IF appointment.userId != userId THEN

        SHOW "Unauthorized"

        RETURN

    ENDIF

    IF appointment.status != "Completed" THEN

        SHOW "Review allowed after completion"

        RETURN

    ENDIF

    IF rating < 1 OR rating > 5 THEN

        SHOW "Rating must be 1 to 5"

        RETURN

```

```

ENDIF

reviewId = GenerateUniqueId()

review = {

  id: reviewId,

  appointmentId: appointmentId,

  vetId: appointment.vetId,

  userId: userId,

  rating: rating,

  comment: comment,

  createdAt: CurrentTime()

}

Firestore.Reviews.Set(reviewId, review)

SHOW "Review submitted"

ENDPROCEDURE

```

5.1.6 Veterinarian Verification Algorithm

Step 1: Veterinarian submits basic profile information (optional).

Step 2: Admin manually reviews the provided details.

Step 3: Admin approves or rejects the veterinarian.

Step 4: System updates the veterinarian's verification status.

- **Veterinarian Verification (Pseudo Code)**

```

PROCEDURE VetSubmitVerification(vetId, vetInfo)

  Firestore.Vets.Update(vetId, {

    verificationData: vetInfo,

    verificationStatus: "Pending"

  })

  SHOW "Verification request submitted"

```



```

ENDPROCEDURE

PROCEDURE AdminVerifyVet(adminId, vetId, decision)

  IF decision == "Approve" THEN

    Firestore.Vets.Update(vetId, { verificationStatus: "Approved" })

  ELSE IF decision == "Reject" THEN

    Firestore.Vets.Update(vetId, { verificationStatus: "Rejected" })

  ENDIF

  SHOW "Vet verification updated"

ENDPROCEDURE

```

5.2 External APIs

The Alhewan App uses several external APIs and libraries to support user authentication, real-time data handling, file storage, chat functionality, and an interactive mobile interface described in the Table 5.1 External APIs for Alhewan App .The primary technologies used in the system are listed below:

Table 5.1 External APIs for Alhewan App

Name of API	Description of API	Purpose of Usage
Firebase Authentication API	Firebase Authentication provides secure user login and registration using email and password. It manages identity verification, sessions, and access control.	Used to authenticate customers, veterinarians, and admins. Ensures only authorized users can access app features such as booking appointments, uploading payments, and viewing dashboards.
Firebase Firestore API	Firestore is a cloud-based NoSQL database that supports real-time storage, queries, and synchronization across devices.	Stores user profiles, pet data, appointments, payment records, chat messages, and reviews. Provides real-time updates for chat and appointment status.
Firebase Storage	Firebase Storage securely stores	Used for uploading and storing

API	images and documents such as screenshots and profile images. It integrates with Firebase Authentication for access control.	payment screenshots, pet photos, and profile pictures. Admins use stored screenshots for payment verification.
Firestore Real-Time Listeners (Streams)	Real-time listeners update data instantly whenever it changes in Firestore, without refreshing the page.	Enables the Live Chat System between customers and veterinarians. Messages appear instantly on both sides.
Firebase Cloud Messaging (FCM)	A push notification service for sending alerts to users across mobile devices.	Sends notifications about appointment confirmations, chat updates, payment verification, and reminders.
Flutter Provider / Bloc Library	State management libraries that manage data flow and app state across screens efficiently.	Used to manage login state, user data, appointment updates, and chat flow across the app. Ensures UI updates automatically.
Flutter UI Widgets	Flutter provides built-in Material and Cupertino widgets for building layouts, forms, lists, and navigation.	Used to design all app screens including Login, Pet Profile, Appointment Booking, Payment Upload, and Chat Interface.
Flutter Image Picker Plugin	Allows users to capture images using the camera or select from the gallery.	Used to upload payment screenshots and pet images inside the app.

5.3 User Interface

The following figures illustrate the major user interface screens implemented in the Alhewan App. Figure 5.1 Screenshot of Login Screen. presents the Login Screen, where users securely access their accounts using email and password authentication. Figure 5.2 Screenshot of Sign-Up Screen. shows the Sign-Up Screen, enabling new users both customers and veterinarians to create an account by providing their essential personal information. Figure 5.3 Screenshot of User Home Screen. displays the Home Screen, offering quick navigation to doctors,

pharmacies, food items, ambulance services, and featured veterinary products through a clean and user-friendly layout.

Figure 5.4 Screenshot of Category Screen. represents the Categories Screen, which organizes veterinary products into groups such as Deworming, Vaccines, Pain Relief, Eye/Ear Drops, Skin & Coat, and Supplements, making it easy for users to browse relevant items. Figure 5.5 Screenshot of User Profile Screen. shows the Profile Screen, where users can update their personal information, view their bookings, and sign out from their account. Figure 5.6 Screenshot of Booking Appointment Screen. illustrates the Doctor Detail Screen, providing valuable information such as clinic details, consultation fee, veterinarian bio, and an appointment calendar for selecting available dates and times.

Figure 5.7 Screenshot of My Appointments Screen. highlights the Appointment Booking Screen, allowing users to schedule a consultation by choosing a preferred date and time. Figure 5.8 Screenshot of Admin Home Screen demonstrates the My Appointments Screen, where users can view their Pending, Confirmed, or Completed appointments, along with access to the real-time chat feature for direct communication with the veterinarian.

On the administrative side, Figure 5.9 Screenshot of Admin Side Product Screen. shows the Admin Dashboard Screen, which displays live statistics such as total users, total doctors, products, orders, and bookings. Figure 5.10 Screenshot of Order Management Screen presents the Product & Stock Management Screen, enabling administrators to view products, update stock levels, edit details, and manage categories.

Figure 5.11 Screenshot of Doctor Verification Management. illustrates the Order Management Screen, where administrators can review all user orders, check their status (Pending, Accepted, Preparing), verify user information, and view item details along with total payment amounts. Finally, Figure 5.12 Screenshot of Appointment Confirmation Screen. represents the Doctor Verification Management Screen, allowing admin users to review submitted veterinarian information, verify documents, approve or reject requests, and maintain a trusted list of verified veterinary professionals. This process improves platform reliability by preventing unverified practitioners from offering services to customers. It also ensures that only qualified veterinarians appear in search results and can receive appointment bookings within the app. User interface implemented in Flutter includes:

5.3.1 Screenshot of Login Screen

The Figure 5.1 Screenshot of Login Screen. allows users to access their accounts securely. It offers email-based login as well as Google authentication for quick sign-in. The screen includes fields for email and password, along with a "Forgot Password" option for account recovery. The minimalistic design ensures a smooth login process and provides a welcoming experience to returning users.

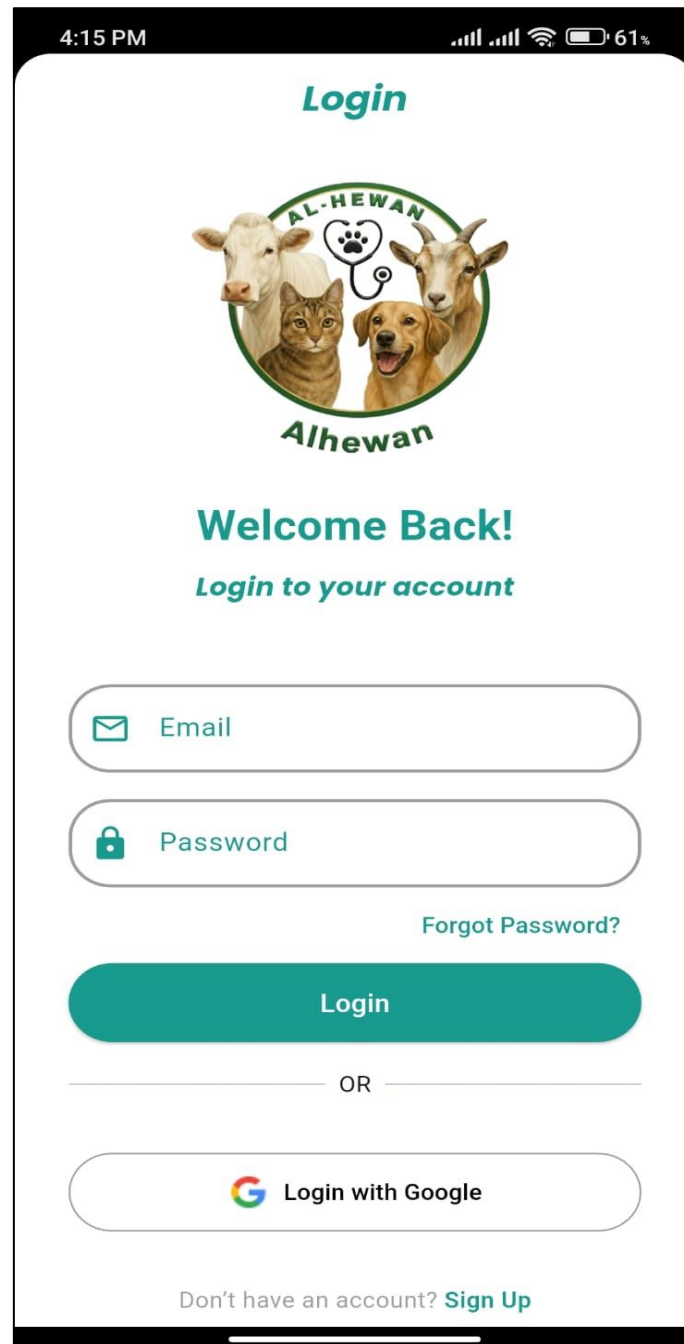
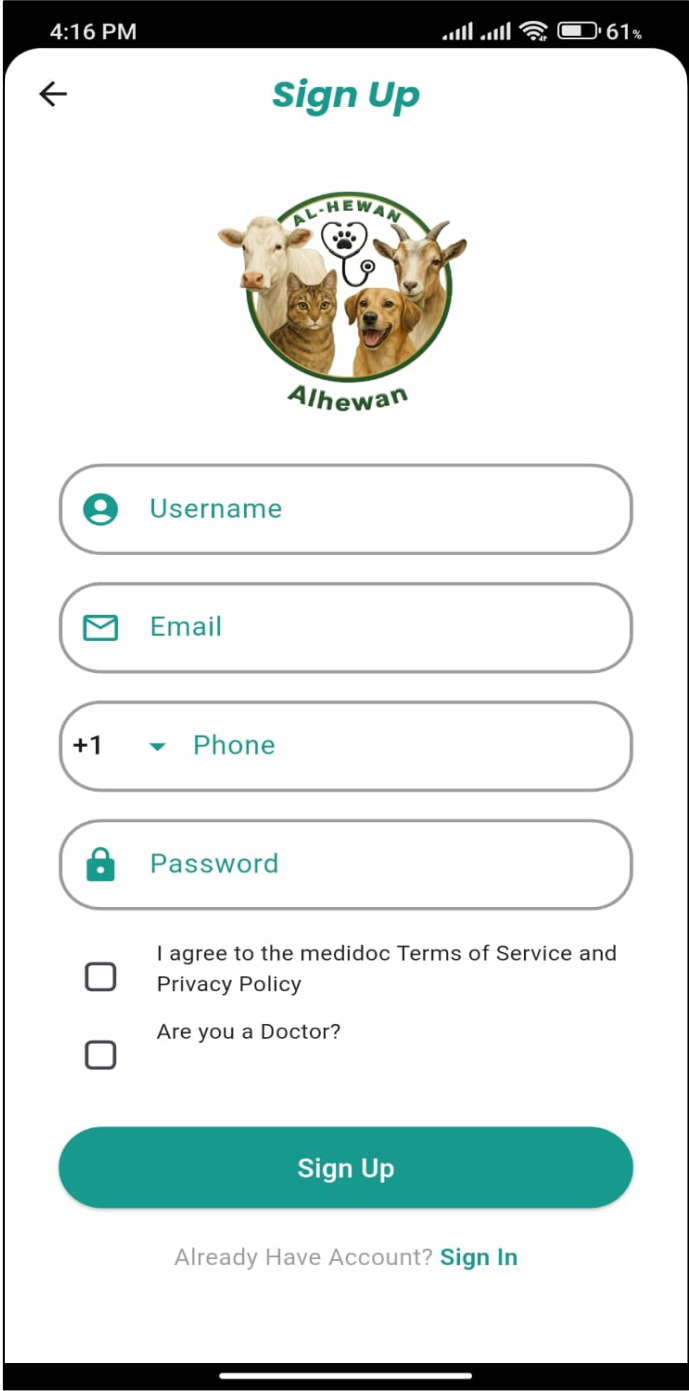


Figure 5.1 Screenshot of Login Screen.

5.3.2 Screenshot of Sign-Up Screen

In Figure 5.2 Screenshot of Sign-Up Screen. enables new users to create an account. It includes fields for entering username, email, phone number, and password. The form also contains checkboxes for agreeing to the terms and conditions and confirming whether the user is a doctor. The interface is designed to make the registration process simple and quick for both customers and veterinarians.



4:16 PM 61%

← **Sign Up**

ALHEWAN
Alhewan

Username

Email

+1 Phone

Password

☐ I agree to the medidoc Terms of Service and Privacy Policy

☐ Are you a Doctor?

Sign Up

Already Have Account? [Sign In](#)

Figure 5.2 Screenshot of Sign-Up Screen.

5.3.3 Screenshot of User Home Screen

In Figure 5.3 Screenshot of User Home Screen. the home screen serves as the main dashboard for users. It includes a search bar for finding doctors, medicines, food items, and articles. It also displays quick shortcuts for Doctor, Pharmacy, Food, and Ambulance services. The screen highlights featured health promotions and shows a list of available veterinarians along with their profiles. The Home screen is designed to provide a smooth and interactive experience by offering easy access to all essential features in one place.

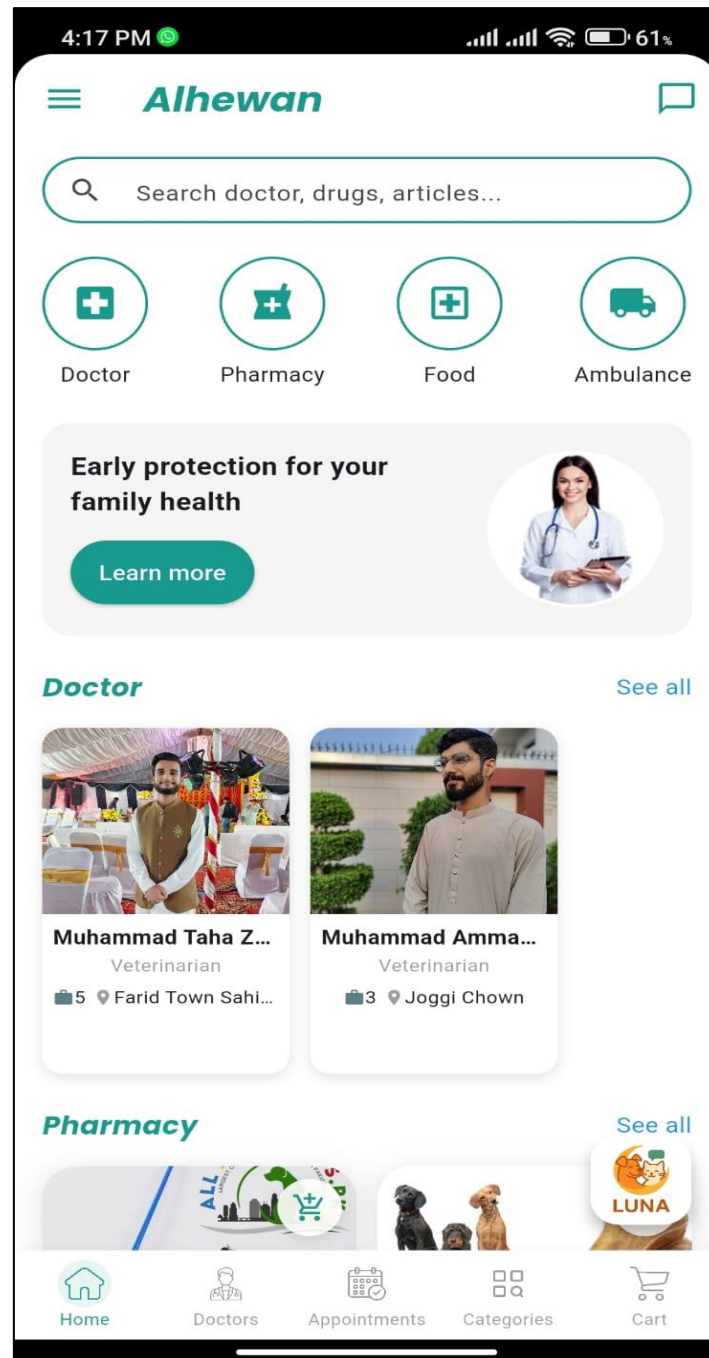


Figure 5.3 Screenshot of User Home Screen.

5.3.4 Screenshot of Category Screen

In Figure 5.4 Screenshot of Category Screen., this screen displays various product categories available in the app, such as Deworming, Vaccines, Pain Relief, Skin & Coat, Eye/Ear Drops, and Supplements. Each category is represented with a simple and meaningful icon for easy navigation. Users can quickly browse relevant pet medicines or products by selecting the category of their choice. The interface is clean, organized, and user-friendly.

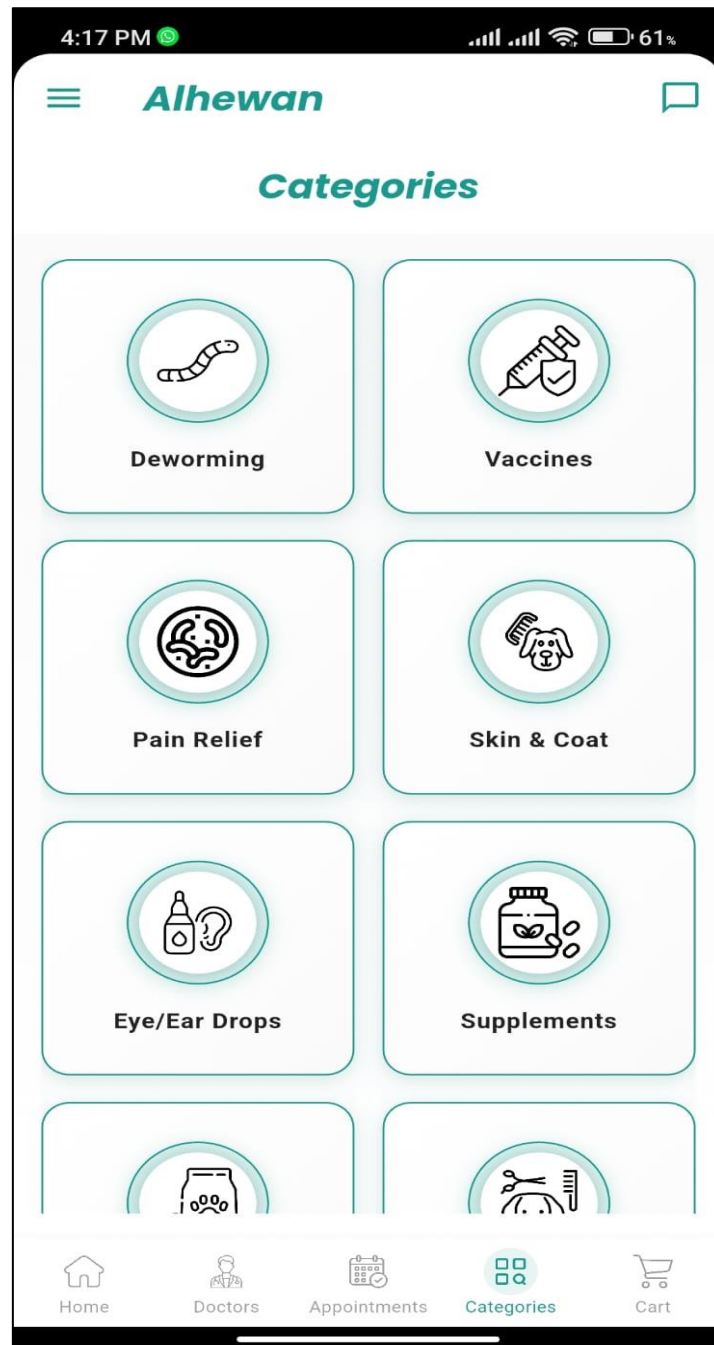


Figure 5.4 Screenshot of Category Screen.

5.3.5 Screenshot of User Profile Screen

In Figure 5.5 Screenshot of User Profile Screen., the Profile screen shows the user's personal details including their username, email, and profile picture. The user can update their profile, manage account information, view appointments, or log out. The layout is simple and intuitive, making it easy for users to customize their personal information and navigate through account-related options.

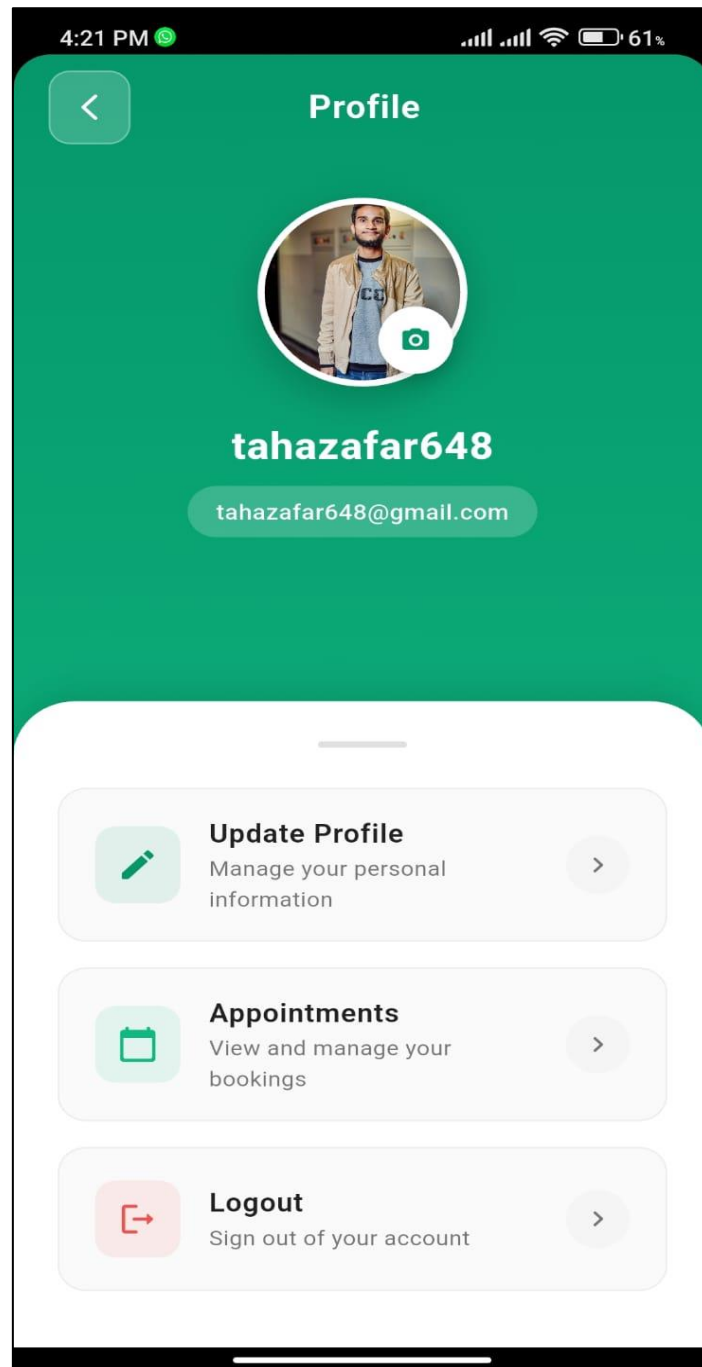


Figure 5.5 Screenshot of User Profile Screen.

5.3.6 Screenshot of Doctor Detail Screen

In Figure 5.6 Screenshot of Booking Appointment Screen., the Doctor Detail screen provides complete information about the selected veterinarian. It includes the doctor's profile picture, name, rating, clinic address, consultation fee, and additional details such as bio and experience. Users can also browse available days and select a preferred appointment date from the integrated calendar. This screen helps users make informed decisions before booking their consultation.

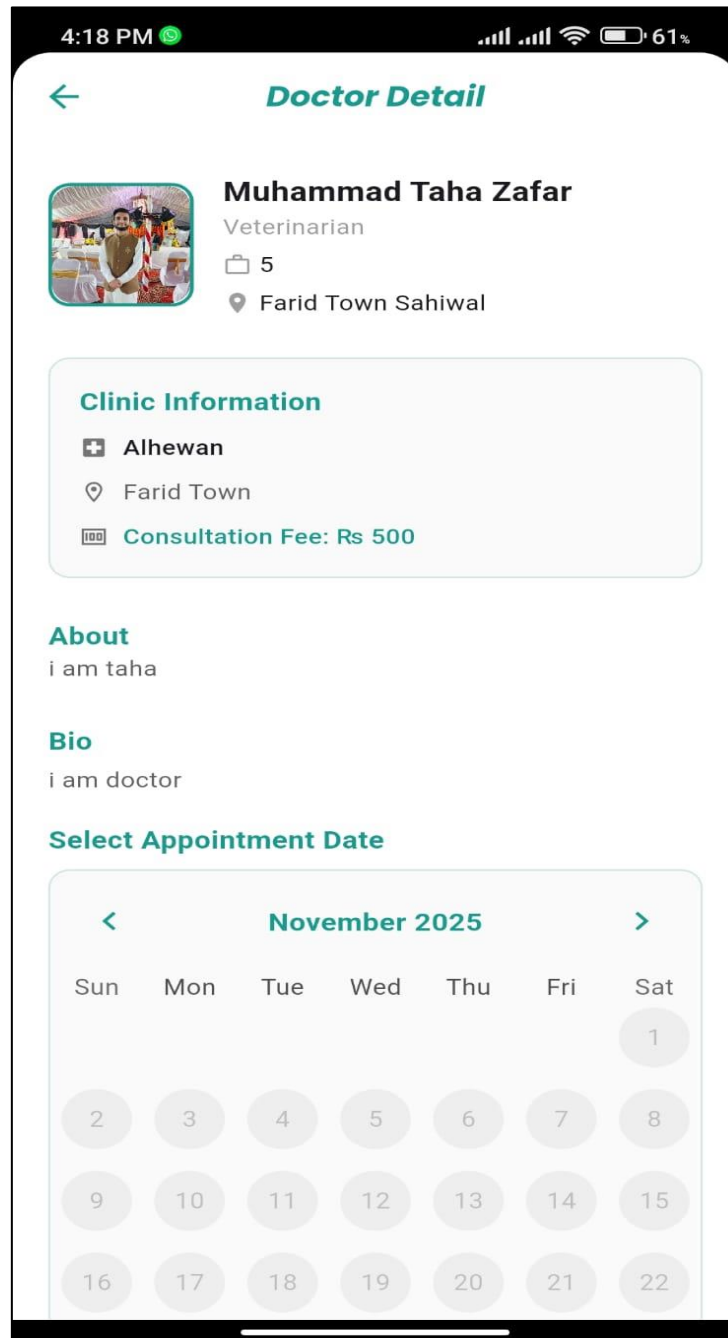


Figure 5.6 Screenshot of Booking Appointment Screen.

5.3.7 Screenshot of Booking Appointments Screen

In Figure 5.7 Screenshot of My Appointments Screen., this screen displays all upcoming, confirmed, and completed appointments booked by the user. Each appointment card shows the veterinarian's name, specialization, date, time slot, pet issue, and appointment status. The user can also initiate real-time chat with the veterinarian directly from the appointment card. The design ensures that all booking details are clearly visible, allowing users to efficiently manage their scheduled consultations.

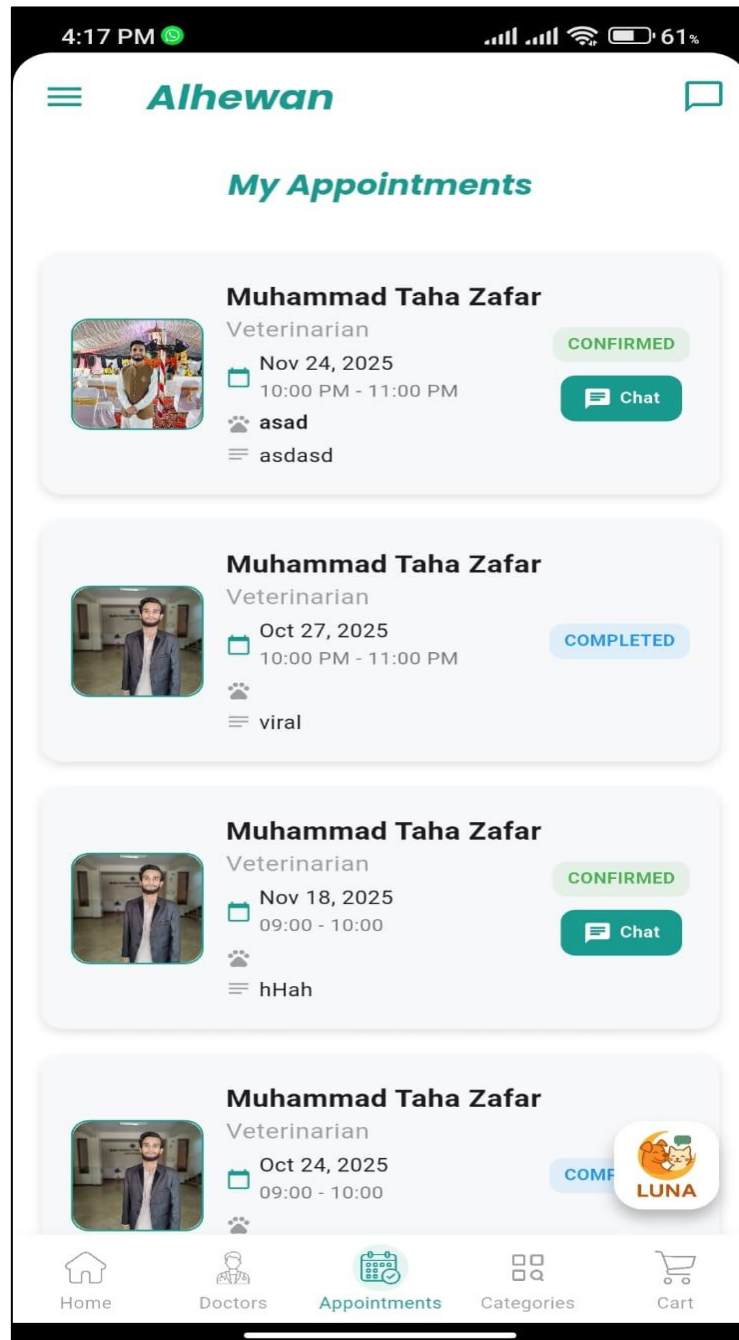


Figure 5.7 Screenshot of My Appointments Screen.

5.3.8 Screenshot of Admin Home Screen

In Figure 5.8 Screenshot of Admin Home Screen, the Admin Dashboard provides a real-time overview of the entire system's performance. It displays live statistics for total users, total doctors, total products, total orders, and total appointments. Each section updates dynamically, helping the administrator understand current activity briefly. This screen acts as the central control panel, giving instant insights into platform engagement, product inventory, and appointment activity. The clean card-based layout supports quick decision-making and enhances administrative efficiency.



Figure 5.8 Screenshot of Admin Home Screen

5.3.9 Screenshot of Admin Side Product Screen

In Figure 5.9 Screenshot of Admin Side Product Screen. the Products & Stock screen enables the admin to manage the entire inventory of veterinary medicines and pet-care supplies. Admins can search products, filter by category or stock status, and view detailed product listings with pricing, supplier information, stock availability, and sold quantities. Quick-adjust buttons allow instant stock updates, while edit and delete options enable modifying product information. This screen plays a crucial role in maintaining accurate inventory levels, preventing stockouts, and ensuring product availability for users.

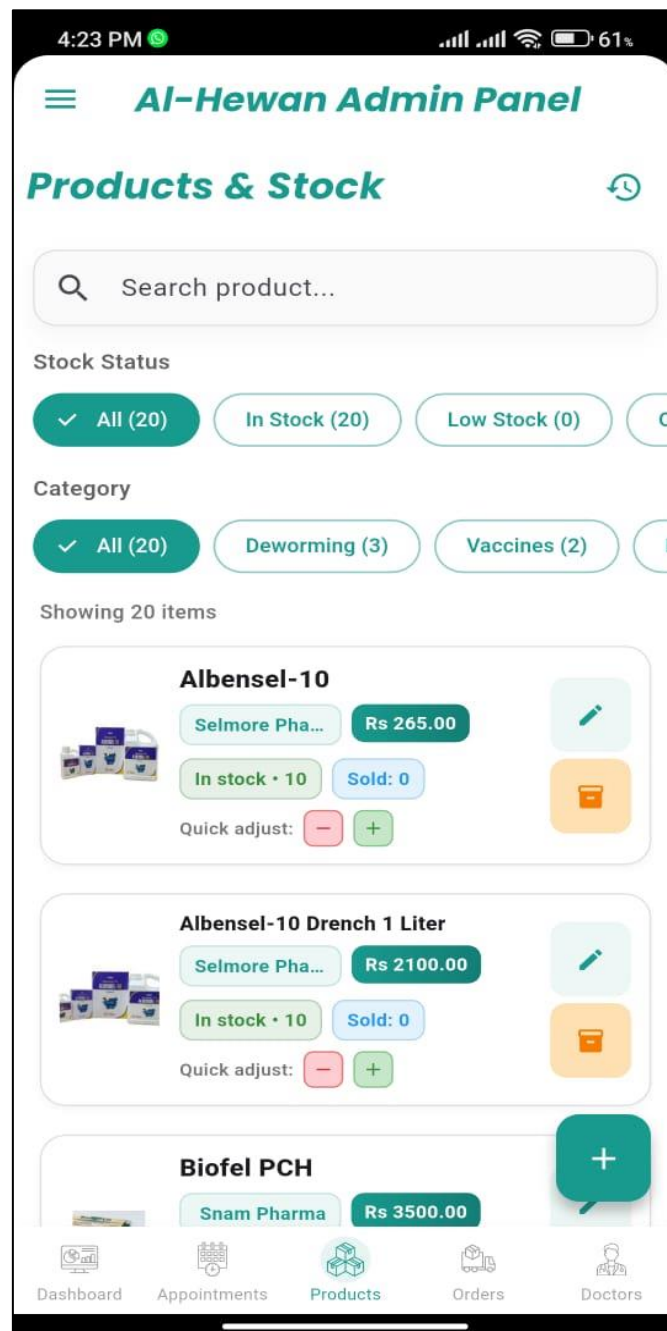


Figure 5.9 Screenshot of Admin Side Product Screen.

5.3.10 Screenshot of Order Management Screen

In Figure 5.10 Screenshot of Order Management Screen, the Order Management screen allows the admin to view, track, and manage all customer product orders in one place. Orders are categorized into different statuses such as *All*, *Pending*, *Accepted*, and *Preparing*, allowing the admin to quickly filter and monitor ongoing activities. Each order card displays complete information, including the customer's name, phone number, delivery address, ordered items, quantity, total amount, and order timestamp. The screen ensures smooth workflow by providing a structured overview of all orders, helping admins manage logistics, monitor pending requests, and process accepted orders more effectively.

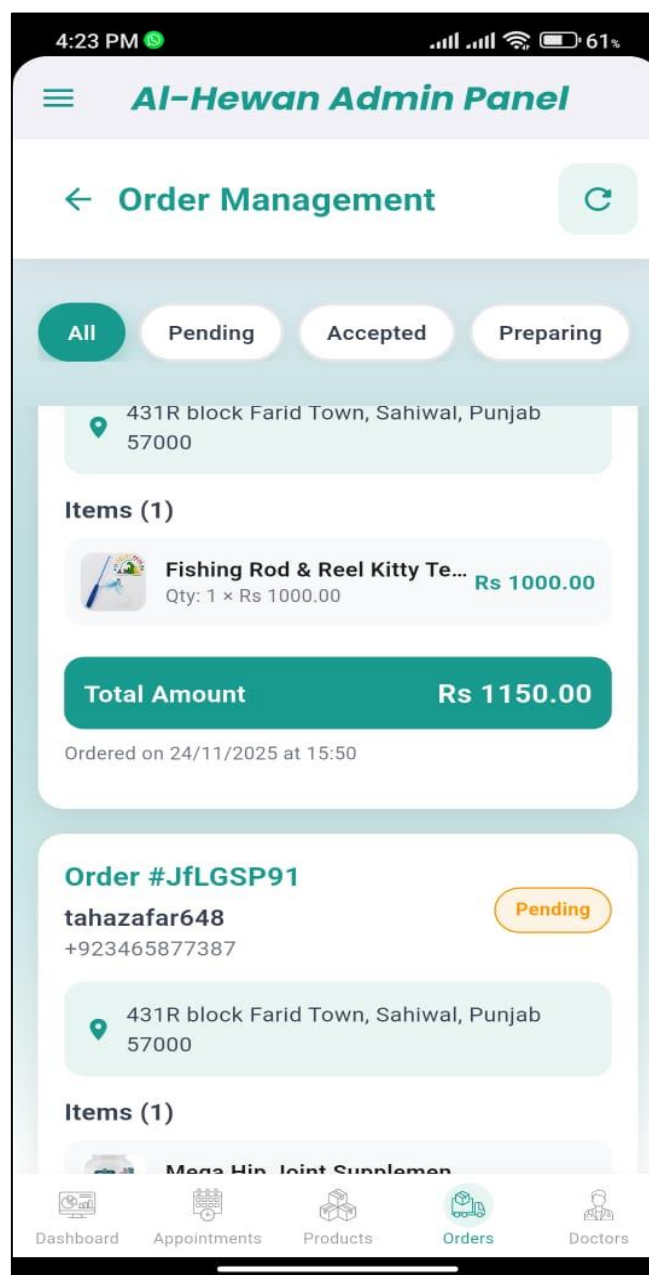


Figure 5.10 Screenshot of Order Management Screen

5.3.11 Screenshot of Doctor Verification Management

In Figure 5.11 Screenshot of Doctor Verification Management. the Doctor Verification Management screen is designed for reviewing and approving veterinarians who apply to join the platform. Admins can filter doctors by *All*, *Verified*, or *Pending* categories. Each doctor card displays essential details such as contact number, email, registration certificate number, consultation fee, and submission date. Using the “View Details” option, the admin can examine documents and verify the doctor’s identity before approval. This feature ensures that only qualified and authentic veterinarians become part of the Alhewan system, maintaining trust and service quality.

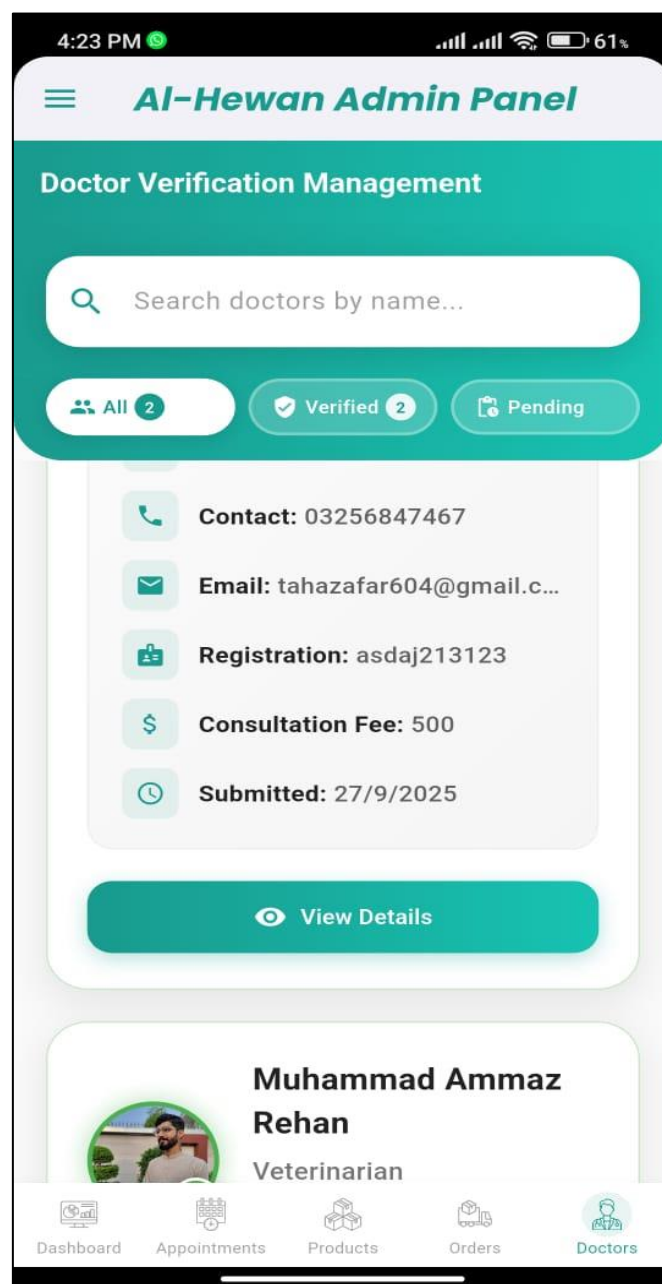


Figure 5.11 Screenshot of Doctor Verification Management.

5.3.12 Screenshot of Appointment Confirmation Screen

In Figure 5.12 Screenshot of Appointment Confirmation Screen. the Confirmed Appointments screen provides a complete overview of all appointments that have been approved by doctors and confirmed for consultation. It shows key statistics such as the total number of appointments and total revenue generated. The search bar helps filter appointments by owner, pet name, or doctor. Each appointment card lists the owner details, pet information, date, time, doctor's name, payment method, and the customer's problem description. This interface allows the admin to efficiently track the progress of confirmed consultations and maintain accurate appointment records.

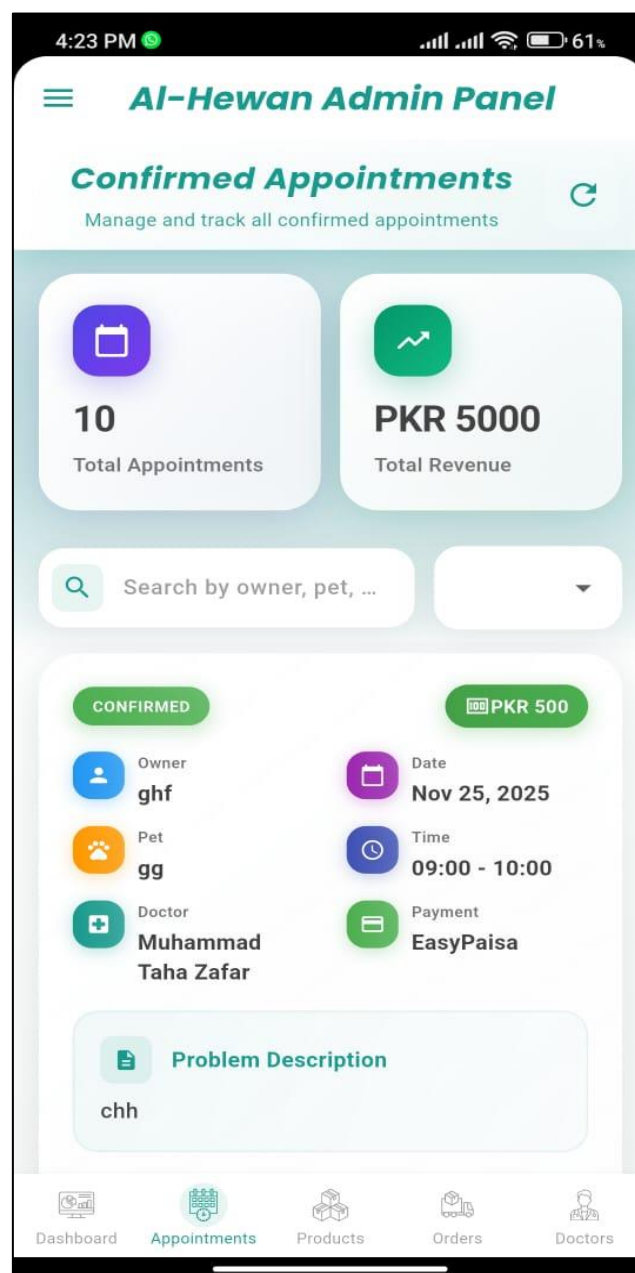


Figure 5.12 Screenshot of Appointment Confirmation Screen.

5.3.13 Screenshot of Doctor Profile Screen

In Figure 5.13 Screenshot of Doctor Profile. this screen shows the veterinarian's profile within the doctor dashboard. It includes the doctor's photo, name, email, and verification status. Veterinarians can update their profile information through the "Edit Profile" option. The layout is clean and emphasizes professionalism by highlighting the verified badge and account settings.

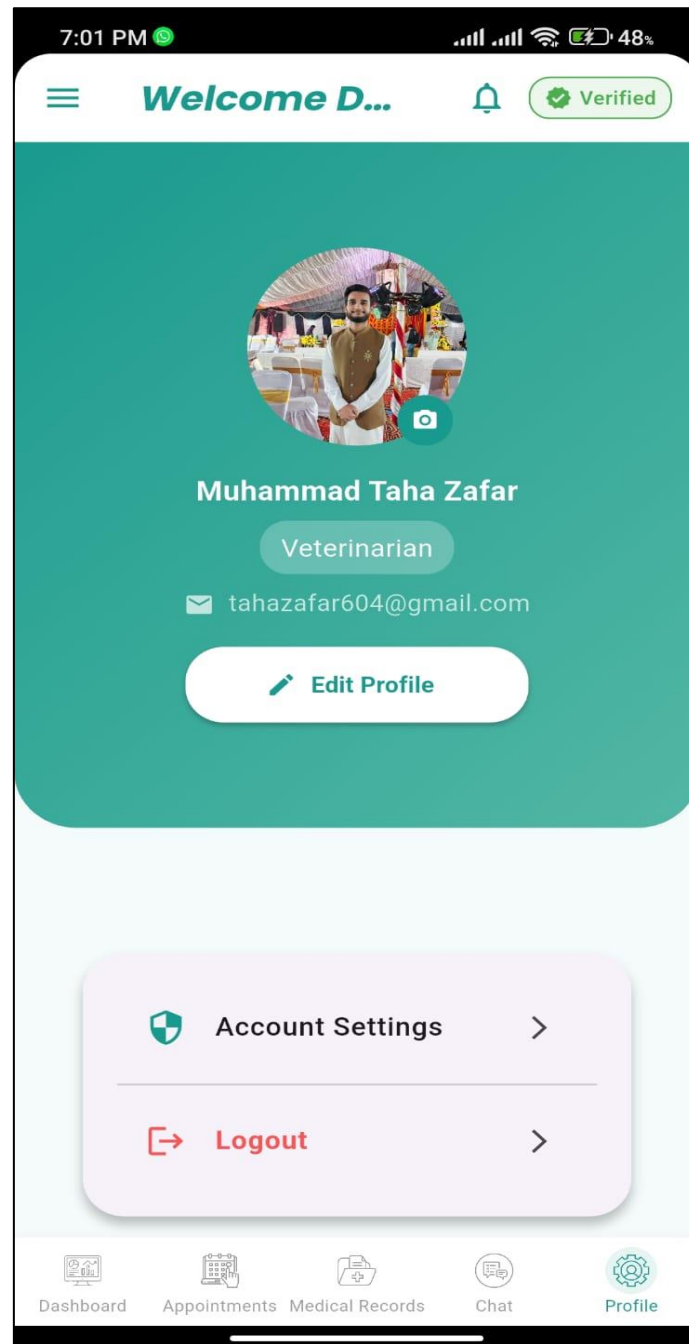


Figure 5.13 Screenshot of Doctor Profile.

5.3.14 Screenshot of Doctor Chat Screen

In Figure 5.14 Screenshot of Doctor chat screen, the chat list screen allows veterinarians to view and manage messages from users. Each chat preview shows the user's profile picture, name, and last message text. The search bar helps doctors quickly locate a conversation. The design focuses on simplicity and ease of communication, supporting real-time interaction between doctors and pet owners.

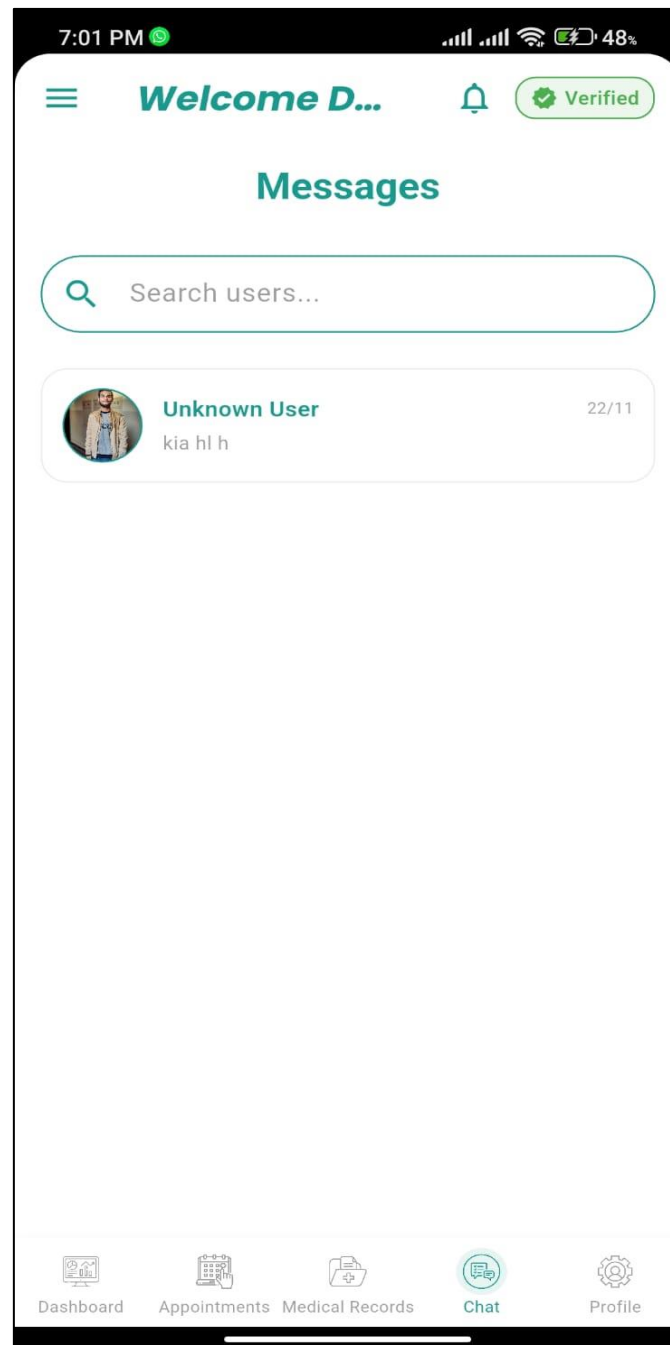


Figure 5.14 Screenshot of Doctor chat screen

5.3.15 Screenshot of Doctor Side Pet History Screen

This Figure 5.15 Screenshot of Doctor side Pet History Screen. presents a complete record of all pets treated by the veterinarian. For each patient, the system displays the pet's name, owner name, and the date of the last visit. The expandable view allows doctors to see full medical history. This feature helps veterinarians track recurring issues, monitor treatment progress, and maintain accurate clinical records.

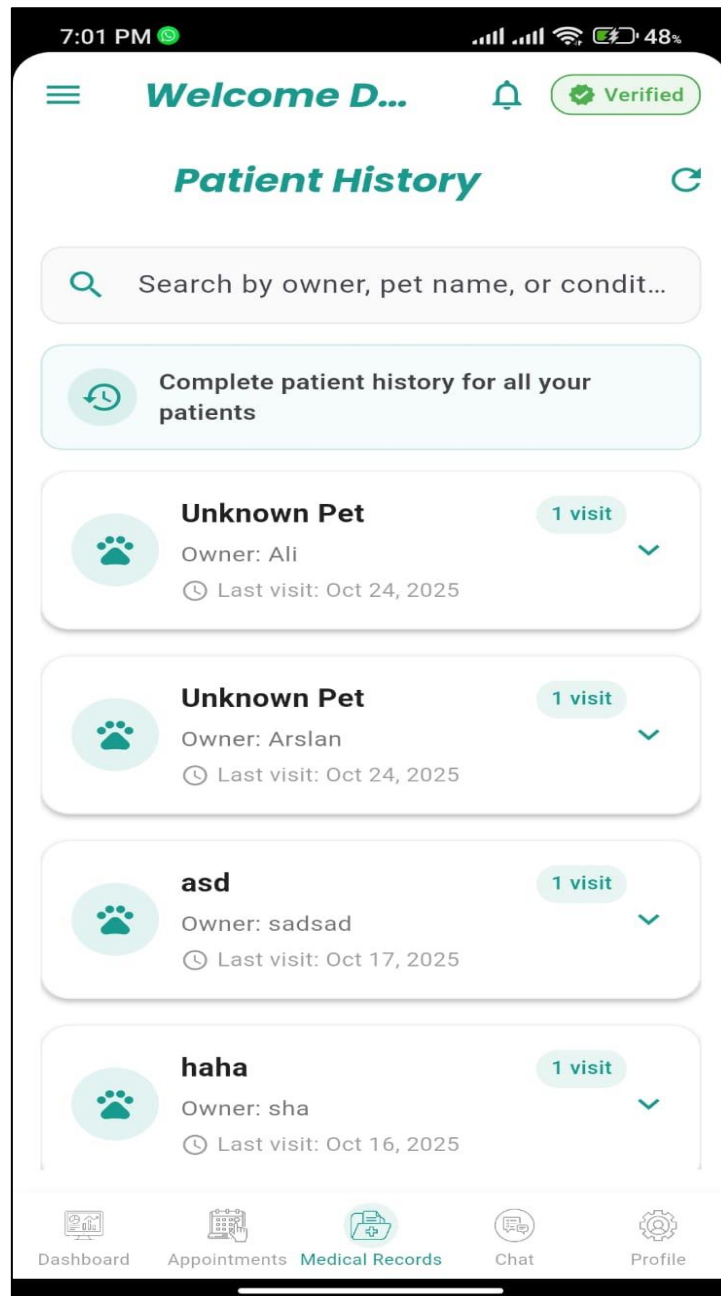


Figure 5.15 Screenshot of Doctor side Pet History Screen.

5.3.16 Screenshot of Doctor Side Appointments Payment Screen

In Figure 5.16 Screenshot of Doctor Side Appointments Payment Screen, this screen provides the veterinarian with a detailed view of a completed appointment. It displays the pet owner's name, consultation type, appointment date and time, service fee, payment method, and the reason for the visit. A payment screenshot is also included for verification. The interface is organized and ensures that the veterinarian can quickly review past consultations for record keeping and future reference.

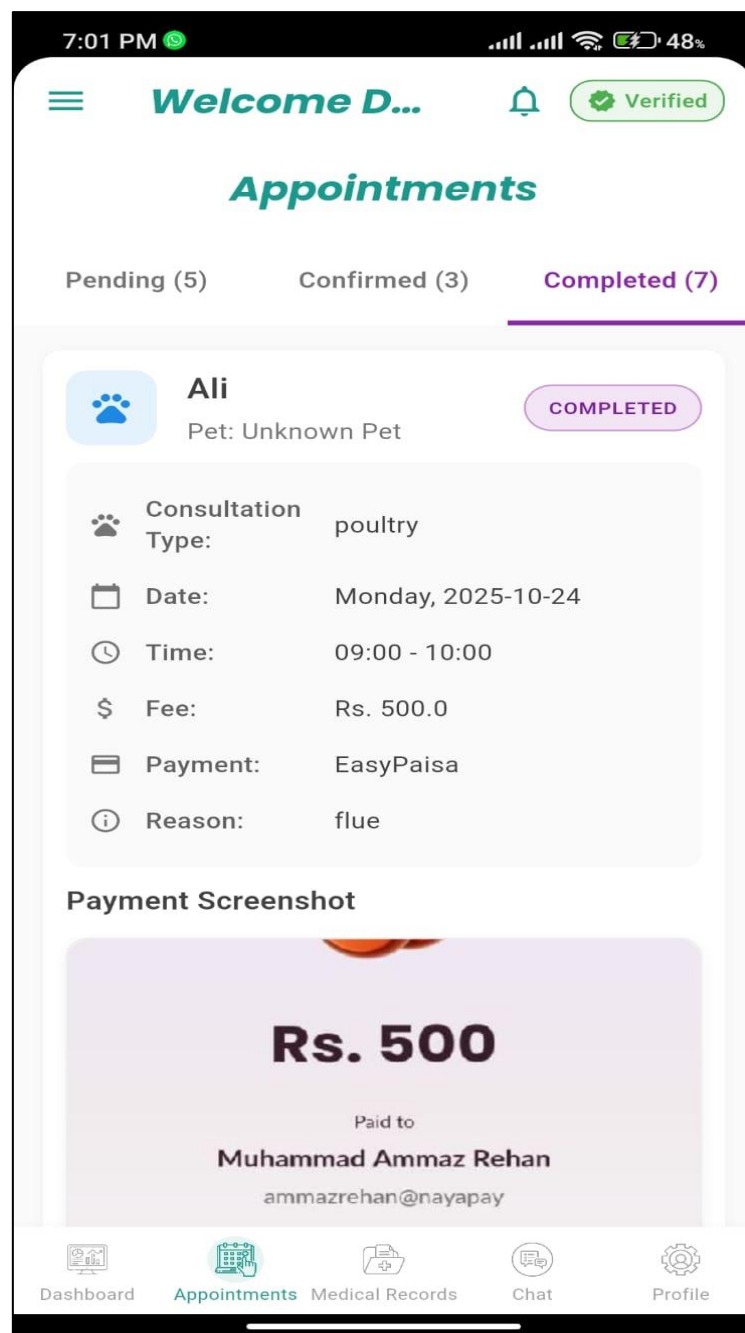


Figure 5.16 Screenshot of Doctor Side Appointments Payment Screen

CHAPTER 6

TESTING AND EVALUATION

6. Testing and Evaluation

The Testing and Evaluation phase represents a vital and foundational component of the Alhewan Veterinary Management System. This phase ensures that every part of the system comprising the User Mobile Application, Veterinarian Mobile Application, and Admin Web Panel operates reliably, securely, and in accordance with the defined functional and non-functional requirements. Since the system integrates multiple interconnected modules such as appointment scheduling, online product purchasing, real-time chat, doctor verification, medical record management, and payment proof handling, comprehensive testing is essential to validate smooth workflow and accurate communication between all components.

The purpose of this phase is to verify system performance, detect inconsistencies, identify functional gaps, and ensure that the platform provides a seamless user experience. Testing confirms the behaviour of critical operations such as user login, appointment creation, payment screenshot uploads, chat synchronization, stock updates, and order processing. It also evaluates system stability under various conditions such as invalid inputs, slow networks, or simultaneous actions from different users.

This phase employs multiple testing methodologies to ensure the overall quality and integrity of the system:

- i. **Manual Testing:** Testing each feature through direct interaction to observe actual behaviour.
- ii. **Unit Testing:** Testing individual functions such as booking appointments, updating stock, calculating totals, and saving chat messages.
- iii. **System Testing:** Validating whether the complete system functions correctly as a unified whole.
- iv. **Functional Testing:** Checking if each feature meets the requirements specified in the SRS.
- v. **Integration Testing:** Ensuring that modules communicate properly and that data flows smoothly among User, Doctor, and Admin systems.

Through these combined testing strategies, the Alhewan system is evaluated for accuracy, performance, usability, reliability, data consistency, and overall system integrity. The Testing and Evaluation phase ensures that the system meets professional quality standards and is ready for real-world use by pet owners, veterinarians, and administrators.

6.1 Manual Testing

Manual testing is one of the most important steps in the evaluation process, as it validates system behaviour from the perspective of real end-users. This method involves manually executing tasks, observing outputs, and confirming whether each part of the system behaves as expected. With manual testing, testers can replicate real-life usage patterns and uncover usability issues that automated tools may overlook.

This phase ensures that the entire workflow of the Alhewan App is functioning correctly. Manual testing is conducted for all three system components.

6.1.1 Manual Testing User Application

The User Application underwent extensive manual testing to ensure that all core functionalities perform accurately and consistently across different scenarios. Each feature was tested individually by executing real-time actions inside the application and comparing the system's responses with the expected outcomes. The primary focus was to validate smooth user experience, reliable data flow, and error-free operation during authentication, product browsing, appointment management, real-time chat, and profile handling.

During testing, all authentication methods such as standard login, signup, and Google Sign-In were evaluated to confirm secure access using Firebase Authentication. Product-related features, including category viewing, product listing, and adding items to the cart, were validated for correctness, ensuring that the system retrieves and displays data accurately from Firebase Firestore. Appointment booking was tested with payment screenshot uploading, verifying successful integration between the user module and Firebase Storage.

Additional testing involved checking appointment history retrieval, ensuring real-time chat synchronization using Firebase Firestore streams, and validating profile editing features as Explained in Table 6.1 Manual Testing User Application. Each test case successfully met its expected output, indicating that all essential modules of the User Application are functioning correctly, providing a stable, user-friendly experience.

Table 6.1 Manual Testing User Application

Test Case	Description	Expected Result	Actual Result	Status
User Authentication	Login and Signup using Firebase Authentication	Successful login	Successful	Pass

Google Sign-In	Login using Google account	Successful authentication	Successful	Pass
View Categories	Display product categories	Correct display	Correct	Pass
View Products	Display product listings	Accurate product loading	Accurate	Pass
Add to Cart	Add selected product to cart	Item added	Item added	Pass
Book Appointment	Book appointment with payment screenshot	Appointment created	Created	Pass
Upload Payment Screenshot	Upload image to Firebase Storage	Upload successful	Successful	Pass
Appointment History	Display user appointments	Complete list displayed	Accurate	Pass
Real-Time Chat	Chat with doctor in real time	Messages synced instantly	Working	Pass
Update Profile	Edit profile and profile picture	Updated successfully	Successful	Pass

6.1.2 Manual Testing Doctor Application

Manual testing for the Doctor Application focused on verifying each core feature used by veterinarians in daily workflow operations as explained in Table 6.2 Manual Testing Doctor Application. The purpose of this testing was to ensure stable performance, accurate data handling, and smooth interaction with the user application and admin panel. Each test case was executed manually to validate login functionality, appointment management, communication tools, and profile operations. Key areas such as doctor authentication, verification badge display, real-time chat, appointment completion, and viewing uploaded payment proofs were thoroughly inspected. All features were checked for accuracy, responsiveness, and error-free execution under typical usage scenarios. The results confirmed that the Doctor Application performs reliably, effectively supports veterinarians in managing their consultations, and

maintains consistent synchronization with Firebase services, ensuring a seamless experience across all interconnected modules.

Table 6.2 Manual Testing Doctor Application

Test Case	Description	Expected Result	Actual Result	Status
Doctor Authentication	Login using Firebase Auth	Successful login	Successful	Pass
Verified Badge	Display verification badge	Should appear correctly	Displayed	Pass
View Appointments	View Pending, Confirmed, Completed tabs	Accurate listing	Accurate	Pass
Payment Screenshot	View uploaded payment proof	Image displayed clearly	Clear	Pass
Complete Appointment	Change appointment status	Updated successfully	Updated	Pass
Doctor Chat	Real-time chat with users	Smooth messaging	Working	Pass
Patient History	View patient visitation history	Correct data displayed	Displayed	Pass
Update Profile	Edit personal and clinic details	Updated successfully	Successful	Pass
Notifications	Receive chat & appointment alerts	Alerts visible	Received	Pass

6.1.3 Manual Testing Admin Panel

The Admin Panel plays a central role in managing the entire Alhewan app, and therefore its testing was conducted with high precision to ensure operational reliability. Manual testing for the Admin Panel as described in Table 6.3 Manual Testing Admin Panel focused on validating all core administrative functionalities, including login authentication, doctor verification

workflows, product management operations, order tracking, appointment monitoring, and dashboard analytics. Each feature was tested step-by-step to verify that the system responds accurately to admin actions and maintains consistency throughout the workflow. Testing began with the secure login process to ensure that only authorized administrators could access the dashboard. The doctor verification module was thoroughly tested to confirm that approval or rejection actions update the database correctly and reflect changes instantly on the doctor's application. Product management functionalities including adding, editing, and deleting items were checked to ensure inventory data syncs properly with the user application. Order management and appointment monitoring modules were tested to ensure the admin can view real-time updates for appointment status, order stages (such as Pending, Accepted, Preparing), and user activity. Additionally, the dashboard's statistical overview was tested to verify that counts refresh dynamically and reflect accurate system data. Across all test cases, the Admin Panel performed consistently, producing accurate outputs with no major issues.

Table 6.3 Manual Testing Admin Panel

Test Case	Description	Expected Result	Actual Result	Status
Admin Login	Secure login to web dashboard	Successful login	Successful	Pass
Verify Doctor	Approve or reject doctor requests	Function works accurately	Working	Pass
Manage Products	Add, edit, delete products	Updated correctly	Updated	Pass
Order Management	Update order preparation & status	Works properly	Accurate	Pass
Appointment Monitoring	View all appointments	Accurate display	Correct	Pass
Dashboard Stats	Real-time updates for counts	Updated instantly	Working	Pass

6.2 System Testing

The System Testing phase ensures that all modules of the Alhewan app work smoothly together in a combined and controlled environment. This testing focuses on validating the behaviour of interconnected features such as authentication, appointment flow, chat system, payment verification, doctor approval, dashboard updates, and order processing. During this phase as described in Table 6.4 System Testing for Alhewan App , the complete workflow from the user booking an appointment to the doctor receiving it and the admin verifying related updates was tested thoroughly. Each module was executed in real-time conditions to confirm that the system responds accurately, maintains smooth communication, and handles data consistently across all platforms. The results show that all integrated components performed as expected, ensuring a reliable and user-friendly experience.

Table 6.4 System Testing for Alhewan App

Test Case	Expected Result	Actual Result	Status
System Startup	All modules load	Loaded successfully	Pass
Real-Time Chat	Smooth communication between user & doctor	Accurate	Pass
Payment Verification	Correct display of payment screenshot	Working	Pass
Appointment Flow	User → Admin → Doctor synchronization	Functional	Pass
Dashboard Updates	Real-time statistics update	Accurate	Pass
Doctor Verification	Admin approval handles correctly	Verified	Pass
Order-to-User Flow	Order created → Admin updates status	Working	Pass

6.3 Unit Testing

6.3.1 Unit Test Appointment Booking

Unit testing for the Appointment Booking module as explained Table 6.5 Unit Test Appointment Booking was carried out to verify that each individual input and function behaves correctly before integration with other components. This test focuses on validating input fields such as date, time, appointment details, and payment screenshot uploads. The aim was to ensure that valid appointments are saved properly, missing or incorrect data is identified, and appropriate error messages are displayed. By testing scenarios like valid bookings, missing screenshots, and overlapping time slots, the system's reliability and error-handling capabilities were confirmed. The successful test results indicate that the appointment booking feature is stable, accurate, and ready for integration with other modules such as notifications and payment verification.

Table 6.5 Unit Test Appointment Booking

Input	Attribute	Expected Result	Actual Result	Status
Valid appointment data	Date, time, details, and screenshot	The appointment should be saved successfully	The appointment was saved successfully	Pass
Appointment without screenshot	Screenshot field is null	An error message should be displayed	An error message was displayed	Pass
Invalid appointment slot	Overlapping time slot	The slot should be unavailable	The slot was marked as unavailable	Pass

6.3.2 Unit Test Product Stock Management

The Product Stock Management as in the Table 6.6 Unit Test Product Stock Management module was thoroughly tested to ensure that all inventory-related operations performed accurately and consistently across the system. Unit testing focused on validating stock updates, error handling, and product editing functions. The tests confirmed that when valid quantities were entered, the system successfully updated product availability without delays. Similarly, attempts to insert negative or incorrect stock values were correctly identified, and the system displayed appropriate error alerts, preventing invalid data from being stored. Editing operations, including price changes and product image updates, were also verified to ensure that modifications were accurately saved and reflected instantly across the application. Overall,

the results confirmed that the stock management logic is stable, secure, and fully aligned with the expected functional requirements.

Table 6.6 Unit Test Product Stock Management

Input	Condition	Expected Result	Actual Result	Result
Valid Stock	Quantity $\geq$ 1	Update successful	Stock quantity updated successfully	Pass
Negative Stock	Quantity $<$ 0	Error alert	Error alert was displayed to the user	Pass
Edit Product	Change price/image	Product updated	Product details were updated successfully	Pass

6.3.3 Unit Test Real Time Chat

The Real-Time Chat module underwent unit testing to validate the reliability and responsiveness of the communication features used by customers and veterinarians as in the Table 6.7 Unit Test Real Time Chat . Testing scenarios included sending and receiving messages, blocking empty or invalid messages, and loading previous chat history. The chat system demonstrated smooth, instant message delivery with real-time synchronization, ensuring uninterrupted communication. The system also correctly prevented the sending of blank messages, maintaining the integrity of conversations. Additionally, viewing past messages was tested and confirmed to load accurately without delays, ensuring users can access complete chat history whenever required. These unit test results verify that the chat module is efficient, responsive, and supports seamless real-time interaction across the platform.

Table 6.7 Unit Test Real Time Chat

Input Scenario	Expected Result	Actual Result	Result
Send / Receive Message	Instant messaging	Messages were sent and received instantly	Pass
Empty Message	Block empty message	System blocked sending of empty message	Pass
View Chat History	Messages loaded	Previous chat messages loaded successfully	Pass

6.4 Functional Testing

Functional testing, as summarized in Table 6.8 Functional Testing, verifies that every feature of the Alhewan Veterinary Management System works according to the defined requirements. This phase ensures that functions across the User App (Customer), Doctor/Veterinarian App, and Admin Panel behave correctly under normal and alternative scenarios. Core features such as user login/signup, veterinarian verification, appointment booking, payment proof uploading, product browsing, cart and order processing, real-time chat, and medical record access were tested individually to confirm reliability, accuracy, and smooth user interaction.

Special attention was given to interactive workflows such as appointment creation, payment proof image uploading, chat messaging, and admin approvals, because these directly affect usability and trust. The test results confirmed that authentication operates securely, appointments are created correctly with required validation, uploaded payment proofs are stored and available for review, chat messages synchronize in real time, and admin operations (product/order management) execute successfully without functional errors. Overall, functional testing validated that the system meets its functional requirements and provides a stable experience for all user roles.

Table 6.8 Functional Testing

Test ID	Description	Expected Result	Actual Result (Observed)	Status
01	User Login & Signup	Secure authentication: user can register and log in successfully	User registration completed; login works with correct credentials; invalid credentials show error message	Pass
02	Veterinarian Verification (Admin Approval)	Verification request is reviewed; status updates correctly (Approved/Rejected)	Admin reviewed request; status changed to Approved/Rejected correctly; veterinarian profile shows updated verification status	Pass
03	Appointment Booking	Appointment is created successfully with correct vet, pet, and time details	Appointment created with selected vet, pet, and slot/time; appointment appears in customer and vet appointment lists	Pass
04	Payment Proof Upload	Proof image uploads successfully; stored and retrievable for review	Payment proof image uploaded and stored; admin/vet can view proof when reviewing payment	Pass

05	Product Browsing	Products load correctly by category with accurate details	Categories displayed; product list loads with correct name, price, and stock; product details open correctly	Pass
06	Cart & Orders	Cart updates correctly; order is created with correct totals and items	Add/remove items updates cart totals; order placed successfully; order items and totals match cart	Pass
07	Real-Time Chat	Messages send/receive instantly and remain consistent for both users	Customer ↔ Vet messages delivered instantly; chat history remains visible; message status updates correctly	Pass
08	Medical Records Access	Veterinarian can view pet/customer history and records correctly	Vet can open pet profile and view medical history/records; records load without missing data	Pass
09	Admin Product Panel	Admin can Add/Edit/Delete products; changes reflect in store immediately	Admin added/edited/deleted products successfully; changes reflected in product browsing results	Pass
10	Admin Order Panel	Admin can update order status; status reflects correctly to customer	Admin updated order status (placed→ shipped→ delivered/cancelled); customer sees updated status correctly	Pass

6.5 Integration Testing

Integration testing in **Table 6.9 Integration Testing** ensures that all modules within the system work together seamlessly and maintain proper data flow across the User App, Doctor App, and Admin Panel. Since the system relies heavily on interconnected workflows such as a user booking an appointment, admin verifying payment proof, or doctors updating appointment status this phase tested how well these modules communicate and synchronize with each other. Key integrated flows were tested, including User → Admin → Doctor synchronization, payment screenshot upload → admin review, admin approval → doctor verification badge, admin product updates → user store display, and bi-directional real-time chat between doctors and customers. Each test confirmed that data moves accurately between all system components and that updates are reflected instantly wherever needed. The successful results demonstrate

that the system maintains full consistency and real-time updates across all applications, ensuring a smooth operational environment.

Table 6.9 Integration Testing

Test Case	Description	Expected Output	Actual Result	Status
User Booking → Doctor View	Appointment visible in doctor panel	Synced correctly	Appointment was displayed correctly in the doctor panel	Pass
Screenshot Upload → Admin Review	Admin sees image	Accurate	Screenshot was successfully visible to the admin	Pass
Admin Approval → Doctor Badge	Verified badge displayed	Working	Verified badge was displayed after admin approval	Pass
Admin Adds Product → User App	Appear instantly in user store	Updated	Product appeared instantly in the user application	Pass
Chat Sync	User ↔ Doctor messages	Real-time sync	Messages were synchronized in real time	Pass
Appointment Completion	Doctor updates → User sees completed	Accurate	Appointment status was updated and shown as completed	Pass

CHAPTER 7

CONCLUSION AND FUTURE WORK

7. Conclusion and Future Work

7.1 Conclusion

The Alhewan app was developed to provide a modern, efficient, and user-friendly digital solution for veterinary services. The system consists of three fully functional modules: the User Application, the Veterinarian Application, and the Admin Panel, all of which work together to streamline the entire veterinary service workflow.

The application allows users to book appointments, upload payment screenshots, chat directly with veterinarians, purchase pet products, and maintain medical records. Veterinarians can manage appointments, access detailed patient histories, communicate with clients through real-time chat, and update their profiles. The Admin Panel supports doctor verification, product management, order tracking, and overall system supervision.

Using Flutter, Firebase Authentication, Cloud Firestore, Firebase Storage, and real-time database features, the system achieves a robust, scalable, and secure architecture. Comprehensive testing including manual testing, unit testing, functional testing, system testing, and integration testing shows that the system performs reliably and meets the required functional specifications.

Overall, the Alhewan app successfully transforms traditional veterinary processes into a digital and streamlined platform, improving accessibility, accuracy, and service quality for users, doctors, and administrators alike.

7.2 Limitations

Despite its successful implementation, the system has a few limitations:

- The application currently does not include real-time audio or video consultation features.
- The payment process is manual, requiring users to upload screenshots instead of processing payments automatically.
- The system depends heavily on Firebase services, which may require upgrades for large-scale traffic.

7.3 Future Work

The system can be further enhanced with the following key features:

- **Online Consultation System**

A real-time online consultation feature, including audio and video communication, can be added to improve remote diagnosis and support.

- **Inventory Management Module**

A complete inventory management system can be implemented for the admin to efficiently manage product stock, pricing, and availability.

- **Multi-Language Support**

The application can offer multiple language options such as Urdu, English, and Arabic to make the system more accessible and user-friendly for a wider audience.

7.4 Final Remarks

The Alhewan app has demonstrated that digital transformation in veterinary services can significantly improve accessibility and management. The system provides a strong foundation for future enhancements, allowing it to scale into a more comprehensive and commercially viable platform. With the planned future updates, the system has the potential to become a leading solution in the veterinary service industry.

References

- [1] Abrahamsson, P., Salo, O., Ronkainen, J., & Warsta, J. (2017). Agile software development methods: Review and analysis. Springer.
- [2] [Veterinary Software Built to Reclaim Time and Connect with Clients](#)
- [3] [Vetstoria | Veterinary Online Booking, Payments, and Websites](#)
- [4] [Otto: Veterinary Workflow Platform](#)
- [5] [Online Appointment Scheduling for Veterinary Practices](#)
- [6] Alaoui, A., & Gahi, Y. (2022). A mobile-based health consultation system using cloud technologies. *International Journal of Computer Science and Information Security*, 20(3), 45–55.
- [7] Google. (2023). Flutter documentation. <https://docs.flutter.dev>
- [8] Google Firebase. (2023). Firebase Authentication documentation. <https://firebase.google.com/docs/auth>
- [9] Google Firebase. (2023). Cloud Firestore documentation. <https://firebase.google.com/docs/firestore>
- [10] Google Firebase. (2023). Firebase Storage documentation. <https://firebase.google.com/docs/storage>
- [11] Google Firebase. (2023). Firebase Cloud Messaging documentation. <https://firebase.google.com/docs/cloud-messaging>
- [12] Milewski, A. (2020). *Software testing essentials: Foundations, techniques, and practices*. Wiley.
- [13] Omar, W., & Malik, H. (2022). Cloud-based data synchronization for mobile apps: Challenges and best practices. *International Journal of Advanced Computer Science*, 13(1), 100–118.
- [14] Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
- [15] Sommerville, I. (2016). *Software engineering* (10th ed.). Addison-Wesley.
- [16] Sultan, T. (2021). Secure appointment booking systems using cloud-based architecture. *Journal of Information Systems & Technology Management*, 18(3), 112–125.
- [17] Zia, R., & Khan, J. (2021). Real-time chat system implementation using Firebase real-time database. *International Journal of Mobile Computing*, 9(1), 14–25.